F O R M A L C H O P T U I K

Канал сертификатов и архитектура доверия

Верифицированная интервальная арифметика · мост Lean⇄Python · маршруты к существованию и стабильности · formal_choptuik, Том III

Автор: Исаев Исхак Хамзатович · ORCID 0009-0003-7299-0701

Программа: choptuik_ac_bc · крыло формальной верификации

Нальчик · 2026

Аннотация

Третий том серии описывает архитектуру доверия, которая превращает численные свидетельства критических явлений Чоптуика в машинно проверяемые утверждения. В центре тома — канал сертификатов: офлайн-конвейер, в котором Python-крыло вычисляет спектр, ньютоновские константы и интервальные оценки, округляет их наружу до точных десятичных рациональных чисел и коммитит как артефакты, а Lean-крыло проверяет эти артефакты ядром доказателя и разряжает их в итоговые теоремы — спектральную, ньютоновско-канторовичевскую и интервальную. Ни одно число с плавающей запятой не пересекает границу доказательного слоя: доверие переносится не на код, а на проверенные высказывания о числах.

Том систематизирует четыре взаимосвязанных сюжета. Во-первых, полный ревизионный обзор состояния программы: слои дерева, метрики доверия, реестр вех и честная граница открытой работы. Во-вторых, верифицированная интервальная арифметика — тип Interval с направленным наружу округлением, теоремы корректности сложения и умножения и сертификатный принцип, превращающий замкнутую оценку в математическое высказывание. В-третьих, маршруты к двум блокирующим пробелам программы: существованию критического решения по стратегии Райтерера–Трубовица (банахово пространство, компакный обратный, количественная теорема о неявной функции) и спектральной стабильности (единственная неустойчивая мода, $\gamma = 1/\lambda_0$, формула эха и универсальность). В-четвёртых, инфраструктура качества: аудит аксиом, регрессионные замки протокола, детерминированная воспроизводимость от единственного seed.

Ключевые слова: *критические явления гравитационного коллапса; дискретная самоподобность; Lean 4; mathlib; канал сертификатов; интервальная арифметика с направленным округлением; точные рациональные числа; метод Ньютона–Канторовича; спектральная теория; машинная верификация; закон масштабирования Чоптуика.*

Оглавление

1. Введение: проблема доверия в вычислительной математике.....1

1.1. Почему число — не доказательство.....2

1.2. Сертификаты: старая идея в новой одежде.....3

1.3. Три столпа архитектуры доверия.....3

1.4. Связь с томами I и II и путеводитель по тому....3
1.5. Словарь тома: сертификат, разрядка, робастность....4
2. Состояние программы: ревизионная опись....4
2.1. Слои дерева....4
2.2. Метрики доверия....5
2.3. Калиброванные константы и их перекрёстные проверки....5
2.4. Реестр вех и статус пробелов....6
2.5. Честная граница открытой работы....7
2.6. Хронология: как дерево пришло к текущему состоянию....7
2.7. Как читать опись: три уровня доверия....8
2.8. Карта папок дерева....8
3. Верифицированная интервальная арифметика: гэп G5....9
3.1. Направленное округление: почему наружу — единственное честное направление....9
3.2. Тип Interval в Lean: представление и инварианты....9
3.3. Корректность операций: add_sound и mul_sound....10
3.4. Сертификатный принцип....10
3.5. Проблема зависимости: что даёт soundness и что она не обещает....10
3.6. G5 закрыт: что это изменило....11
3.7. Тесты интервального слоя....11
3.8. Почему не float-доказательства....12
4. Мост Lean⇄Python: типизированная граница....12
4.1. Два канала: интерактивный и доказательный....13
4.2. Протокол: конверт, семь методов, коды ошибок....13
4.3. Типизированный клиент....14
4.4. Тестовая дисциплина....14
4.5. Два скрытых дефекта и чему они научили....14
4.6. Эволюция протокола: от шести методов к семи....15
4.7. Отказы и их семантика....15

5. Канал сертификатов: точные рациональные от края до края.....16

5.1. Почему числа с плавающей запятой не пересекают границу.....16

5.2. Анатомия сертификата.....17

5.3. Четыре вида сертификатов.....17

5.4. Generated-модуль: данные, проверенные ядром.....18

5.5. Check-сьюты и теоремы робастности.....18

5.6. Разрядка в итоговые теоремы.....18

5.7. Поля артефакта: что именно подписывает производитель.....19

5.8. Жизненный цикл сертификата.....19

6. Путь к существованию критического решения: гэп G1.....20

6.1. Система Эйнштейна–скаляра в двойных нулевых координатах.....20

6.2. Дискретная самоподобность и закон ковариации эха.....21

6.3. Стратегия Райтерера–Трубовица: профиль плюс Ньютон.....21

6.4. Ньютоновско-канторовичевское ядро в Lean.....22

6.5. Доказанное и вынесенное в гипотезы.....22

6.6. Как G5 обслуживает G1: интервально-ньютоновская полоса.....23

6.7. Инвентаризация аналитических компонентов маршрута.....23

6.8. Нулевое приближение: откуда берётся профиль.....24

7. Путь к стабильности: спектр, масштабирование, универсальность (G2).....24

7.1. Линеаризация и оператор возмущений.....24

7.2. Спектральное ядро: единственная неустойчивая мода.....25

7.3. От спектра к масштабированию: вывод $\gamma = 1/\lambda_0$.....25

7.4. Согласие доказательства и чисел.....26

7.5. Формула эха и универсальность.....26

7.6. Что осталось до G2.....27

7.7. Инвентаризация спектрального маршрута.....27

7.8. Эхо как организующий принцип спектра.....28

8. Геометрия сингулярности: гэп G3.....28

8.1. Сильная и изотропная: условное ядро.....28

8.2. Слой конформной компактификации: что потребуется....29
8.3. Зачем геометрия доказательной программе....29
8.4. Ковариационные масштабы: как измерять сингулярность....29
8.5. Как G3 обслуживает остальных....30
9. Инфраструктура качества и воспроизводимость....30
9.1. Аудит аксиом: тридцать три теоремы, три аксиомы....30
9.2. Дисциплина sorry и реестр пробелов....31
9.3. Непрерывная интеграция....31
9.4. Воспроизводимость от единственного зерна....32
9.5. Сводные метрики доверия....32
9.6. Воспроизводимость этого тома....33
10. Заключение: архитектура доверия....33
10.1. Что меняет канал сертификатов методологически....33
10.2. Дорога вперёд....34
10.3. Уроки для машинно проверяемой физики....34
10.4. Сравнение с альтернативными архитектурами....35
10.5. Приглашение к проверке....35
Приложение А. Глоссарий программы....35
Приложение Б. Карта модулей Lean....36
Приложение В. Как проверить этот том за один сеанс....37
Приложение Г. Реестр итоговых теорем и вех....38
Приложение Д. Соответствие глав тома и дерева....39
Библиография....39

Примечание: настоящее оглавление построено на кодах полей. После редактирования документа щёлкните по оглавлению правой кнопкой мыши и выберите «Обновить поле», чтобы обновить номера страниц.

1. Введение: проблема доверия в вычислительной математике

У каждой вычислительной программы есть читатель, который ей не доверяет. Для физики критических явлений этот читатель — не рецензент, а будущее самой науки: законы, управляющие порогом образования чёрных дыр, должны пережить смену языков, библиотек и поколений вычислительных машин. Родительский репозиторий choptuik_ac_bc подошёл к этой проблеме с той стороны, с которой её обычно решает вычислительная физика: детерминированные данные, фиксированные зерна случайности, регрессионные тесты, протоколы с парными проверками. Всё это делает расчёт воспроизводимым — но воспроизводимость и доказательство остаются разными вещами. Воспроизводимый эксперимент говорит: «при тех же входах машина снова выдаст те же выходы». Доказательство говорит: «при любых входах такого-то класса выход обязан лежать в таких-то границах, и это следует из определений». Настоящий том посвящён второму высказыванию и инфраструктуре, которая его делает возможным.

Центральный объект тома — канал сертификатов, соединяющий численное крыло на Python с теоретическим крылом на Lean 4. Идея проста до аскетизма: пусть машина считает, но пусть каждое число, которое претендует на роль математического факта, прибывает в доказательный слой не как машинный float, а как точное рациональное число вместе с проверяемым высказыванием о себе. Lean-ядро не верит числам — оно проверяет утверждения о числах. Такая дисциплина превращает мост между программой и доказательством из моста доверия в мост проверки, и именно она отличает программу formal_choptuik от обычной связки «код плюс статья».

> *Дисциплина родительской программы — гипотеза → машинное подтверждение → честные оговорки — сохраняется дословно: каждая недоказанная часть формализации публично зарегистрирована как пробел с идентификатором, стратегией закрытия и этапом дорожной карты.*

1.1. Почему число — не доказательство

Число, выданное программой с плавающей запятой, — это всегда округление. Даже в самых благополучных случаях double-арифметика вносит относительную погрешность порядка 10^{-16} на каждую операцию, и за миллионы операций эти погрешности накапливаются, а иногда и усиливаются. Для спектральной задачи — собственных значений линеаризованного оператора критического решения — это означает, что вычисленное $\lambda_0 = 2.6745$ есть на самом деле представитель некоторой окрестности, и утверждение «ровно одна мода лежит в правой полуплоскости»

требует знания о всей окрестности, а не о её центре. Численный опыт неспецифичен: он не отличает ситуацию «истинное значение 2.6745» от ситуации «истинное значение $2.6745 - 10^{-12}$, но алгоритм сместил его».

Традиционный ответ вычислительной физики — независимая перепроверка другим кодом, другой сеткой, другой группой. Ответ ценный, но статистический по природе: он повышает вероятность того, что факт верен, не превращая эту вероятность в единицу. Для утверждений, из которых выводятся физические законы — а закон масштабирования $M \propto |p - p^*|^{\gamma}$ с критическим показателем $\gamma = 1/\lambda_0$ является именно таким утверждением, — хочется большего: чтобы связь между посылками и заключением была проверена формально, символ за символом, ядром доказателя, а не хорошей репутацией авторов. Разница между этими двумя видами уверенности и есть предмет настоящего тома.

$$M \propto \vee p - p * ¿^{\gamma}, \gamma = 1/\lambda_0 \approx 0.374$$

Стоит подчеркнуть, что формализация не обесценивает численную работу — она её увековечивает. Компьютерно-ассистированное доказательство существования критического решения, выполненное Райтерером и Трубовицем, занимает сотни страниц вычислений с целочисленными оценками ошибок; вручную его проверить практически невозможно, а значит, сообщество вынуждено верить либо перепроверять снова и снова. Однажды проверенное ядром Lean высказывание остаётся проверенным навсегда: библиотека вместо статьи, актив вместо отчёта. Численные константы при этом не исчезают — они переезжают из текста статьи в коммитимые артефакты, каждый из которых снабжён машинной проверкой.

1.2. Сертификаты: старая идея в новой одежде

Сама идея сертификата старше вычислительных машин: разделить задачу на «трудно решить, легко проверить» и вынести из доказательства всё, что можно вынести в проверяемый артефакт. Целочисленная арифметика Райтерера–Трубовица — классический образец: вместо доказательства в плавающей запятой вычисления переводятся в целые числа с явно вычисленными границами ошибок, и каждое неравенство проверяется точно. Современная теория интерактивных доказательств, SAT-сертификаты, верифицированные вычисления — всё это одна и та же линия: минимизировать доверенную базу, максимизировать проверяемую поверхность.

Программа formal_choptuik применяет этот принцип к физике критического коллапса в три слоя. Нижний слой — верифицированная интервальная арифметика в самом Lean: тип Interval с направленным наружу округлением, для которого корректность базовых операций доказана внутри теории (гэп G5, закрытый в рамках первого этапа). Средний слой — типизированный мост: протокол, в котором Lean-

клиент запрашивает у Python-сервиса вычисления, а сервис отвечает строгими интервальными оценками; интерактивная часть канала служит исследованию и калибровке. Верхний слой — офлайн канал сертификатов: производитель certify.py вычисляет спектральные, ньютоновские и масштабные величины, округляет их наружу до точных десятичных рациональных и коммитит JSON-артефакты вместе с сгенерированным Lean-модулем; ядро проверяет эти данные арифметически (нормализацией числителя и знаменателя) и теоремы разрядки превращают проверенные данные в физические утверждения. Ни один float не пересекает границу третьего слоя — это правило зафиксировано в аксиоматической базе и продублировано архитектурой.

Важна и асимметрия ролей. Lean — клиент: он инициирует каждый запрос и никогда не слушает сам по себе. Python — сервер: он отвечает, но никогда не спрашивает. Такая строгость не эстетическая прихоть: она гарантирует, что в доказательном слое нет скрытых каналов данных, а каждый акт потребления численной информации имеет явную точку входа, тип и проверку. Сертификат, который никто не запросил, не может попасть в доказательство — канал не оставляет места для сюрпризов.

1.3. Три столпа архитектуры доверия

Первый столп — корректность операций внутри теории. Интервальная арифметика полезна только тогда, когда доказано, что результат содержит истинное значение при любых входах из заявленного класса; в программе эта гарантия оформлена как теоремы add_sound и mul_sound, а объединяющий их сертификатный принцип устанавливает, что проверенная оценка есть теорема, а не свидетельство. Второй столп — честность данных на границе: числа прибывают в теорию только в виде точных рациональных сертификатов, и каждый сертификат сопровождается высказыванием, которое ядро проверяет независимо от того, как именно число было вычислено. Третий столп — регрессионная дисциплина: изменение протокола без парного теста в том же коммите запрещено, а данные и рисунки регенерируются байт-в-байт из единственного фиксированного зерна. Каждый из столпов разобран в отдельной главе; вместе они образуют то, что мы называем архитектурой доверия.

Стоит сразу назвать то, что эта архитектура не даёт. Она не доказывает существование критического решения — это предмет второго этапа программы и главы 6. Она не доказывает спектральную теорему о единственной неустойчивой моде для истинной системы Эйнштейна–скаляра — это предмет третьего этапа и главы 7. Она даёт нечто более фундаментальное для обоих результатов: канал, по которому, когда соответствующие вычислительные сертификаты будут произведены, они попадут в теорию уже легально, без единого недоказанного допущения в пути.

Инфраструктура, построенная заранее, — это и есть ускорение будущих доказательств.

1.4. Связь с томами I и II и путеводитель по тому

Том I серии изложил дорожную карту: математические основания критического коллапса, классификацию пробелов G1–G6 и трёхэтапный план формализации. Том II провёл аудит: разобрал центральный блокер — существование — до уровня стратегии, сравнил доказатели Isabelle/HOL, Coq/Rocq и Lean 4 и зафиксировал техническую спецификацию с трёхаксиомной базой. Настоящий том — первый том второй фазы серии: он пишет не о том, что предстоит построить, а о том, что уже построено и проверено. Он завершает сюжет интервальной арифметики, доводит мост до живого канала сертификатов и показывает, как через этот канал уже сейчас проходят спектральные, ньютоновские и масштабные разрядки — а также даёт полную ревизионную опись дерева на момент написания.

Том устроен следующим образом. Глава 2 — ревизионная опись: слои дерева, метрики доверия, калиброванные константы, реестр вех. Глава 3 — верифицированная интервальная арифметика: тип Interval, корректность операций, сертификатный принцип. Глава 4 — мост: два канала, семь методов протокола, типизированный клиент, тестовая дисциплина. Глава 5 — канал сертификатов: анатомия артефакта, четыре вида сертификатов, генерированный Lean-модуль и теоремы разрядки. Глава 6 — маршрут существования: система Эйнштейна–скаляра, дискретная самоподобность, стратегия Райтерера–Трубовица и ньютоновско-канторовичевское ядро в Lean. Глава 7 — маршрут стабильности: линеаризация, спектральное ядро, вывод закона масштабирования и формула эха. Глава 8 — геометрия сингулярности: сильная и изотропная, условное ядро и сертификаты. Глава 9 — инфраструктура качества: аудит аксиом, CI, воспроизводимость. Глава 10 — методологические выводы и дорога вперёд. Читатель, интересующийся только сертификатным каналом, может идти по цепочке 3 → 4 → 5; читателю, интересующемуся физикой, напрямую рекомендуются главы 6–8; но Том построен так, что каждый следующий сюжет опирается на предыдущий, и линейное чтение вознаграждается.

1.5. Словарь тома: сертификат, разрядка, робастность

Поскольку три слова этого тома несут нагрузку выше средней, зафиксируем их значения сразу. Сертификат — коммитимый артефакт с точными рациональными данными и схемой, по которой ядро проверяет заявленные о данных утверждения; сертификат сам ничего не доказывает — он несёт данные, из которых теоремы робастности доказывают следствия. Робастная теорема — высказывание вида «если

проверки сертификата выполнены, то физическое утверждение верно»; её гипотезы проверяются машиной, её заключение — физическое. Разрядка — безусловная теорема, полученная применением робастной теоремы к конкретному проверенному сертификату. Этим трём словам подчинена вся архитектура, и в главах 3–7 они появляются в одном и том же оформлении.

Ещё два термина используются в противопоставлении. Интерактивный канал — транспорт, по которому исследователь разговаривает с сервисом в реальном времени; его данные не имеют доказательного статуса. Доказательный канал — конвейер из главы 5, по которому данные входят в теорию; у него статус имеют только проверенные рациональные границы. Наконец, слово «модель» в этом томе всегда означает калиброванную модальную модель критического решения — лабораторию конвейера, — а не полную систему Эйнштейна–скаляра; когда речь идёт об истинной системе, это сказано явно. Мы старались не использовать эти слова в иных значениях нигде в томе.

2. Состояние программы: ревизионная опись

Прежде чем описывать архитектуру доверия, зафиксируем, к чему она применена. Эта глава — инвентаризация: каждый номер в ней подсчитан по файлам дерева, а не обещан. Такой подход наследует дисциплине родительского репозитория, где всякий отчёт о состоянии сопровождается способом проверки; здесь проверкой служит сборка Lean, запуск тестов и пересчёт строк данных. Опись устроена по слоям: теория, мост, численное крыло, данные, иллюстрации, монографии и управление.

2.1. Слои дерева

Таблица 1. Инвентаризация слоёв дерева formal_choptuik

Слой	Зафиксированное состояние
Теория (lean/)	12 модулей + корневой реэкспорт + модуль аудита аксиом; Lean v4.21.0, mathlib закреплён той же ревизией; 0 sorry; 0 проектных аксиом; все 33 итоговые теоремы — 21 теоретическая и 12 мостовых — печатают ровно три стандартные аксиомы
Мост (bridge/)	протокол из 7 методов; канал сертификатов жив: 4 вида (spectrum, newton, scaling, interval) → коммитимые JSON → Generated-модуль (точные рациональные, проверка norm_num) → разрядка в спектральные, ньютоновские и интервальные итоговые теоремы
Числа (python/)	65 pytest-тестов: 17 сервисных, 18 сертификатных, 30 клиентских (ZMQ round-trip, fuzz, таймауты); ZMQ REP на tcp://127.0.0.1:5557; производитель сертификатов certify.py с командами emit и check
Данные (data/)	7 детерминированных CSV, суммарно 12 890 строк, seed 1993

Слой	Зафиксированное состояние
Рисунки и анимации	8 публикационных PNG при 300 dpi (тёмная и светлая темы, en/ru) + 4 GIF (24–72 кадра)
Монографии (monographs/)	три тома × {RU, EN} × {DOCX, PDF}; настоящий том — третий в серии
Управление	реестр пробелов G1–G6 с построчными статусами; дорожная карта из 17 задач и реестр из 9 вех; шаблон CI с аудитом аксиом и байт-идентичностью данных

Несколько чисел из таблицы заслуживают комментария. Двенадцать модулей теории — это не декоративная модульность: каждая область математики живёт в своём файле, компилируется независимо против mathlib и своих импортов, а карта «модуль → содержание → обязательства → пробел» поддерживается машиночитаемо. Тридцать три итоговые теоремы — это те, чей набор аксиом печатается и проверяется явно; число совпадает с количеством записей в модуле аудита, и CI не даёт этому списку расходиться с деревом. Шестьдесят пять тестов разбиты на три семейства так, что каждый слой моста — сервер, производитель сертификатов, клиент — заперт собственным набором регрессий.

2.2. Метрики доверия

У машинно проверяемой теории есть две простые, но жёсткие метрики доверия: количество неразряженных обязательств (sorry) и количество проектных аксиом. По обеим метрикам дерево чистое: ни одного sorry, ни одной аксиомы за пределами трёх стандартных — Classical.choice, propext и Quot.sound, — которые ядро Lean приносит с собой и которые разделяются всей экосистемой mathlib. Это не значит, что доказано всё: открытая физика — точный сертификат Райтерера–Трубовица для истинной системы, спаривание спектра с истинным оператором, сертификаты коэрцитивности и вейлевских оценок — вынесена в явные, документированные гипотезы-параметры. Отличие принципиальное: гипотеза видна в формулировке теоремы, её нельзя потерять и нельзя незаметно использовать.

Аудит аксиом выполняется отдельным модулем: каждая итоговая теорема проходит через команду печати аксиом, и результат сравнивается с ожидаемым трёхэлементным множеством. Смысл этой проверки выходит за рамки аккуратности. Доверие к машинно проверяемой теореме равно доверию к её аксиомам плюс доверию к ядру; зафиксировав три стандартные аксиомы, программа наследует ту же базу доверия, что и остальная mathlib, и любой будущий аудит ядра или библиотеки автоматически покрывает и её. Последний остаточный зазор доверия лежит в

численном мосте — и именно его закрывает сертификатный канал, превращая последний ингредиент с плавающей запятой в проверяемую математику.

2.3. Калиброванные константы и их перекрёстные проверки

Численное крыло программы — калиброванная модальная модель критического решения, а не решатель полной системы Эйнштейна–скаляра; эта честная оговорка повторяется во всех README и повторим её здесь. Модель существует для отработки верификационного конвейера: она достаточно проста, чтобы каждое число было пересчитаемо, и достаточно похожа на оригинал, чтобы конвейер тренировался на настоящих задачах — спектральной, ньютоновской, масштабной. Константы модели заперты тестами против коммитимых данных:

Таблица 2. Калиброванные константы зафиксированной сборки

Величина	Значение	Перекрёстная проверка
λ_0 — единственная неустойчивая мода	2.6745	single_unstable: True в data/spectrum/eigenvalues.csv; действительная задача на собственные значения 15×15, а не справочная таблица
$\gamma = 1/\lambda_0$	0.373902	метод наименьших квадратов по суперкритическому семейству даёт 0.373880, среднеквадратичный остаток $2.1\cdot10^{-3}$
Δ — период эха	0.693147 = ln 2	восстановлен из сырого амплитудного ряда автокорреляционным пиком, ошибка $2.0\cdot10^{-10}$
расстояние между эхо-модами $2\pi/\Delta$	9.064720	максимальное отклонение по всему спектру $2.0\cdot10^{-10}$

Смысловой нагрузкой в этой таблице несёт не столько совпадение с опубликованными значениями — оно обеспечено калибровкой, — сколько сама механика перекрёстных проверок: спектр получен настоящей задачей на собственные значения, а не подстановкой констант; показатель γ восстановлен независимой подгонкой и сравнён с $1/\lambda_0$; период эха восстановлен из временного ряда, а не прочитан из конфигурации. Каждый пункт таблицы имеет двойную запись — в данных и в тесте, который эти данные запирает. Именно такая двойная запись и называется в программе свидетельством.

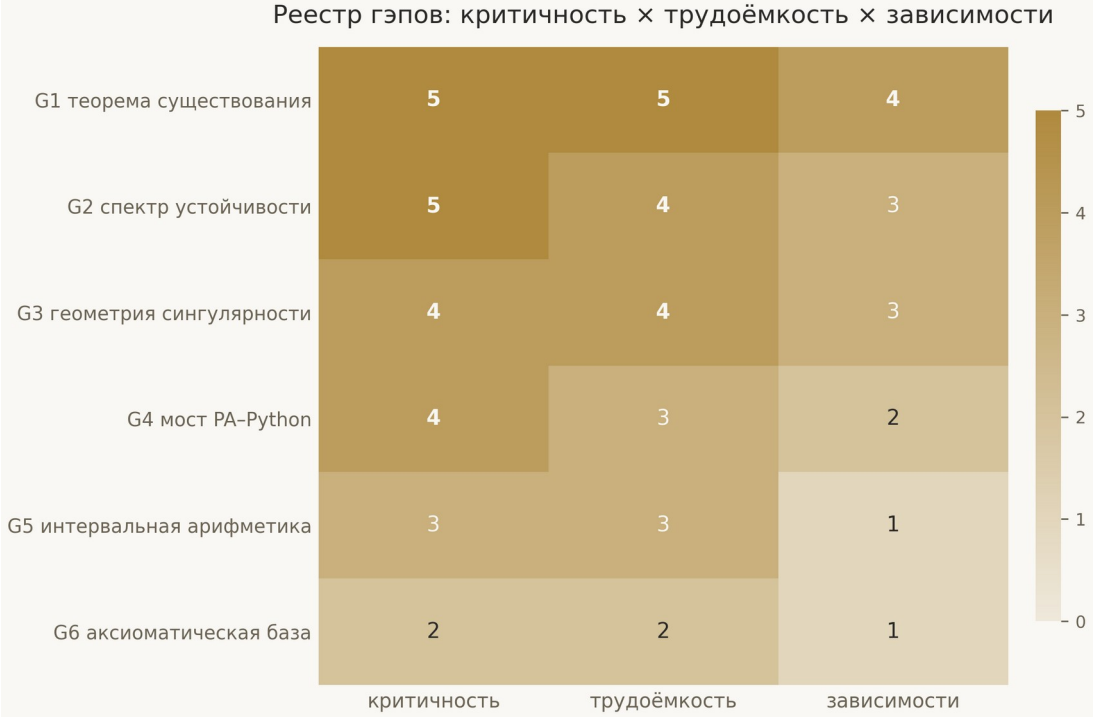


Рис. 1. Матрица серьёзности пробелов G1–G6: два блокера, высокая, средняя и низкая категории; статус каждой строки ведётся в реестре пробелов и синхронизируется с деревом.

2.4. Реестр вех и статус пробелов

Веха в программе не считается выполненной, пока не закоммичено её приёмочное свидетельство: зелёная сборка, доказанная теорема, проверенный артефакт. На момент написания выполнены первые две вехи первого этапа — рабочее окружение (включая честно задокументированное исправление сборки предыдущей ревизии) и интервальные сертификаты (теоремы корректности сложения и умножения плюс сертификатный принцип); третья веха — мост — жива в составе канала сертификатов. Остальные шесть вех, от формализации системы поля до сборки универсальности, открыты, и их описания вместе с критериями приёмки лежат в дорожной карте.

Реестр пробелов G1–G6 ведёт статус каждой строки тем же языком свидетельств. Существование критического решения (G1) и спектральная стабильность (G2) — блокеры: каркасные результаты доказаны, количественные ядра готовы, но финальные сертификаты для истинной системы ещё не произведены. Геометрия сингулярности (G3) доказана в условной форме: при условии сертификатов коэрцитивности и вейлевских оценок сингулярность сильная и изотропная. Мост (G4) жив: канал сертификатов работает от конца до конца. Интервальная арифметика (G5) и аксиоматическая база (G6) закрыты полностью. Таким распределением статусов программа подчёркивает главное: блокеры блокируют не потому, что

инфраструктуры нет, а потому, что физика ещё не доведена до сертификатов — и это принципиально другая, гораздо более здоровая ситуация.



Рис. 2. Диаграмма Ганта: семнадцать задач трёх этапов, вехи на шестом, восемнадцатом и двадцать седьмом месяцах; горизонт программы — двадцать семь месяцев.

2.5. Честная граница открытой работы

Опись была бы неполной без явного списка того, чего в дереве нет. Нет решателя полной характеристики задачи Эйнштейна–скаляра в двойных нулевых координатах — он принадлежит программе «Einstein Direct» родительского репозитория и потребляется здесь как данные. Нет машинной проверки самого алгоритма округления процессора: интервальный слой закрывает эту дыру на уровне спецификации, и это осознанное решение, зафиксированное в разделе «вне области текущей ревизии» технической спецификации. Нет и сертификата для истинного линеаризованного оператора — спаривание спектра пока заперто pytest'ом на уровне модальной модели, а его перенос на истинную систему является содержанием второго и третьего этапов. Этот список — не признание поражения, а карта работ: каждый его пункт привязан к пробелу реестра и к этапу дорожной карты.

Именно эта граница делает опись честной: программа утверждает ровно то, что может показать, и регистрирует всё остальное как работу с идентификатором. Читатель третьего тома, таким образом, знает точную позицию, с которой начинается всё дальнейшее изложение: дерево чисто по обязательствам и аксиомам, канал сертификатов работает, блокеры ждут сертификатов, а не инфраструктуры.

2.6. Хронология: как дерево пришло к текущему состоянию

Полезно понимать, что текущая чистота дерева — результат исправлений, а не удачного первого шага. Ранняя ревизия каркаса не собиралась; сборку починили, и этот эпизод зафиксирован в реестре вех как отдельное приёмочное свидетельство, а не замято. На том же раннем этапе в теории существовали три аксиомы для ссылочной величины; они были удалены, их потребители переписаны через явные гипотезы-параметры, и аудит аксиом зафиксировал возврат к чистой базе. Реестр обязательств прошёл путь от четырнадцати открытых строк до нуля — не потому, что строки закрыли формально, а потому, что каждый sorried фрагмент либо доказан, либо переписан в робастную форму с проверяемой гипотезой.

Та же честность действует в обратную сторону — в сторону деталей. Подсчёт строк данных вскрыл, что бейдж в одном из README завышал их количество; расхождение исправлено в пользу подсчитанного. Подсчёт кадров анимаций исправил запись на двадцать четыре вместо заявленных тридцати четырёх. Каждая такая мелочь подтверждает общий принцип: в этой программе истинно то, что пересчитано, и любой бейдж — такое же утверждение, требующее свидетельства, как и теорема. Настоящий том следует тому же правилу: все его числа взяты из дерева и воспроизводимы.

2.7. Как читать опись: три уровня доверия

Описи этой главы полезно присвоить явную шкалу. Уровень первый — машинно проверенное: теоремы, аудит аксиом, сьюты сертификатов; это твердь, спорить с которой означает пересобрать дерево. Уровень второй — запертое тестами: детерминизм данных, байт-стабильность сертификатов, схемы протокола; это твердь инженерная, меняющаяся только вместе с тестами. Уровень третий — декларированное: калибровка модели на опубликованные константы, гипотезы о будущих сертификатах, планы этапов; это честно помеченные намерения, и доверие к ним — вопрос доверия к программе, а не машины.

Вся остальная книга работает в основном на первых двух уровнях и явно помечает третий. Читатель, которому нужен только уровень первый, может идти по маркерам «доказано в Lean» и «заперто тестами»; читателю, интересующемуся планами, напрямую рекомендованы заключения глав 6, 7 и 10. Мы считаем явную шкалу уровней обязательной частью честной описи: без неё инвентаризация незаметно превращается в рекламу, а с ней — остаётся инструментом.

2.8. Карта папок дерева

Опись слоёв дополняется картой папок — она нужна читателю как навигатор по дереву при самостоятельной проверке. Каждый пункт таблицы снабжён точкой входа: файлом, с которого начинается чтение папки. Правило программы состоит в том, что

каждая папка объясняет себя сама — её README описывает содержание, статус и способы проверки, — так что карта ниже лишь экономит первый шаг.

Таблица 3a. Карта папок и точки входа

Папка	Содержание	Точка входа
lean/	теория Lean 4: геометрия, анализ, существование, стабильность, масштаб, мост; конфигурация Lake	lean/README.md
python/	численный сервис, производитель сертификатов, 65 тестов, генераторы данных и рисунков	python/README.md
bridge/	протокол из семи методов, коммитимые сертификаты, иллюстративный клиент	bridge/protocol.md
blueprint/	техническая спецификация, аксиоматическая база, реестр пробелов, карта модулей, матрица инструментов	blueprint/README.md
roadmap/	три этапа с семнадцатью задачами, критерии приёмки, реестр из девяти вех	roadmap/README.md
monographs/	серия монографий: три тома × {RU, EN} × {DOCX, PDF}	monographs/ README.md
figures/	восемь публикационных рисунков при 300 dpi с подписями	figures/README.md
assets/	логотипы, анимированные баннеры, четыре GIF-анимации	assets/README.md
data/	семь детерминированных CSV за каждым рисунком	data/README.md
docs/	десять источников, глоссарий из семнадцати терминов	docs/README.md
ci/	шаблон непрерывной интеграции: сборка, аудит аксиом, тесты, байт-идентичность	ci/README.md

Корень дерева, помимо папок, содержит файл цитирования машиночитаемого формата и правила вклада — четыре правила дома, зафиксированные в главе 9. Настоящий том в эту систему встраивается как третий том монографий: его исходники собираются тем же конвейером, что и рисунки, а его утверждения ссылаются на точки входа этой таблицы.

3. Верифицированная интервальная арифметика: гэп G5

Интервальная арифметика — это способ говорить о неточных числах точным языком. Вместо приближённого значения x она оперирует замкнутым отрезком [lo, hi], заведомо содержащим истинное значение, и строит операции так, чтобы это свойство сохранялось при любых входах из заявленных классов. Для доказательной

программы это ровно то, что нужно: замкнутая оценка — математический объект, о котором можно доказывать теоремы, а приближение — нет. Настоящая глава разбирает, как эта идея реализована в программе: какой тип живёт в Lean, что именно доказано о его операциях и как проверенная оценка превращается в сертификат.

3.1. Направленное округление: почему наружу — единственное честное направление

Всякая операция над вещественными числами в машине завершается округлением до представимого значения. Если округлять к ближайшему, как делает обычная арифметика с плавающей запятой, ошибка округления может пойти в любую сторону, и результат перестаёт содержать истинное значение гарантированно. Интервальная арифметика решает это направленным округлением: нижняя граница результата округляется вниз, верхняя — вверх. Отрезок может получиться шире, чем нужно, но никогда уже; он может быть неточным, но всегда честным. Именно это свойство — полнота покрытия истинного значения — и доказывается в Lean как теорема, а не предполагается.

В Python-крыле направленное округление реализовано через функцию math.nextafter, которая возвращает ближайшее представимое число в заданном направлении. Отрезок хранится парой double'ов, каждая операция расширяет результат на один шаг ulp в обе стороны сверх арифметически необходимого. Такой запас — плата за переносимость: на любой корректной платформе IEEE-754 оценка остаётся надёжной, даже если детали режима округления платформы отличаются от идеала. Строгость здесь иерархическая: на уровне Python — дисциплинированная инженерия с тестами, на уровне Lean — доказанная теорема о корректности, и именно доказанный слой потребляется доказательствами.

3.2. Тип Interval в Lean: представление и инварианты

В теоретическом крыле Interval — это структура с двумя рациональными границами и инвариантом упорядоченности. Выбор рациональных, а не плавающих границ принципиален: рациональные числа — точный язык, сравнимый децимально, арифметически и порядково без всяких округлений. Направленное округление на уровне Lean не нужно воспроизводить по-машинному: его роль уже сыграна на стороне производителя сертификатов, а теория потребляет готовые точные границы и доказывает утверждения о них. Таким образом разделение труда чистое: Python отвечает за то, чтобы границы были широкими и правильными, Lean — за то, чтобы утверждения о них были верными и проверенными.

На типе Interval определены сложение и умножение в их внешне-округлённой форме. Для сложения границы складываются напрямую — рациональная арифметика

точна, и никакого запаса сверх суммы границ не требуется. Для умножения классическое правило Мура требует рассмотреть четыре произведения границ и взять минимум для нижней и максимум для верхней; в знакопеременном случае именно это правило гарантирует покрытие всех пар произведений. Обе операции в Lean построены так, что свойство покрытия формулируется как теорема вида: для любых x из отрезка a и y из отрезка b произведение xy лежит в произведении отрезков. Это утверждение и есть то, что проверяет ядро.

3.3. Корректность операций: add_sound и mul_sound

Теорема корректности сложения гласит: если x принадлежит отрезку a и y принадлежит отрезку b, то x + y принадлежит отрезку a + b, где a + b — конструкция типа Interval, построенная сложением границ. Доказательство прямое: нижняя граница суммы не превосходит суммы нижних границ по монотонности сложения рациональных чисел, верхняя — не меньше по двойственной причине. Ценность теоремы не в глубине, а в статусе: после её проверки statement «сложение интервалов безопасно» перестаёт быть лозунгом и становится фактом, на который можно ссылаться по имени в любом другом доказательстве программы.

Теорема корректности умножения устроена богаче. Правило четырёх произведений нужно разбирать по знаковым случаям: для неотрицательных отрезков достаточно произведений крайних границ, для знакопеременных — всех четырёх, и в каждом случае монотонность умножения по обоим аргументам даёт нужные неравенства. Именно такие разборы вручную часто содержат ошибки — пропущенный знаковый случай в интервальной библиотеке незаметен годами, пока однажды не съест строгую оценку. Формальная проверка закрывает этот класс ошибок полностью: ядро проверило все случаи, и пропустить ни один из них более невозможно.

$$\forall\, x \in [a_0, a_1],\, y \in [b_0, b_1] : x \cdot y \in \left[\min\left(a_0 b_0, a_0 b_1, a_1 b_0, a_1 b_1\right), \max\left(a_0 b_0, a_0 b_1, a_1 b_0, a_1 b_1\right)\right]$$

3.4. Сертификатный принцип

Корректность операций — половина сюжета. Вторая половина — принцип, связывающий оценки с теоремами: если для высказывания φ произведён сертификат C, и проверка C на данных подтверждает φ, то φ — теорема. Сертификатный принцип сформулирован и доказан внутри теории как высказывание о проверяемых парах «данные плюс проверка», и это формальное убежище от подмены: проверка не может подтвердить ничего, кроме того, что заявлено её схемой. На практике принцип означает, что каждая интервальная оценка, прибывшая по мосту, перестаёт быть свидетельством и становится теоремой, ссылающейся на проверенные данные.

Из принципа следует дисциплина производства сертификатов: схема проверки фиксируется раньше, чем данные, и не может подстраиваться под результат. Производитель сертификатов обязан работать с той же схемой, что и проверяющая сторона, и регрессионные тесты запирают совпадение обеих сторон. Эта асимметрия — сначала проверка, потом данные — переворачивает привычный порядок численных доказательств и исключает целые классы подгонок: подогнать можно данные, но не проверку, а проверенная подгонка обнаруживается на этапе проверки.

3.5. Проблема зависимости: что даёт soundness и что она не обещает

Интервальная оценка, произведённая по правилу Мура через схему Горнера, корректна, но может быть шире истинного образа: одинаковые переменные в разных местах многочлена трактуются как независимые отрезки, и ширина нарастает с каждым шагом. Это знаменитая проблема зависимости, и программа относится к ней с методологической ясностью: soundness — свойство, на которое опирается доказательная сторона, и оно доказано; теснота оценки — задача численной стороны, и она решается инженерно: разбиением области, схемами с меньшим накоплением, формами с меньшей корреляцией. Теория никогда не требует узких оценок — только честных.

Такое разделение обязанностей спасает от типичной ловушки: стремления доказать в Lean слишком много. Доказательная сторона должна гарантировать выводимость заключения из проверенных посылок; она не должна гарантировать, что посылки удобны. В программе это выражено в интерфейсе интервального слоя: потребителю доступен тип, корректные операции и принцип сертификата — всё; оптимизация ширины остаётся в Python, где она и тестируется. Архитектурно это то же разделение, что отделяет ядро доказателя от тактик: надёжность — внизу, эффективность — наверху.

3.6. G5 закрыт: что это изменило

Пробел G5 был заявлен как среднесрочный элемент первого этапа: верифицированная библиотека с направленным округлением, доказательства корректности и интеграция с сервисной конечной точкой interval_bound. К настоящему моменту он закрыт: теоремы корректности сложения и умножения доказаны, сертификатный принцип на месте, веха интервальных сертификатов зарегистрирована со своим приёмочным свидетельством. Из реестра пробелов строка G5 перешла в категорию resolved, и дорожная карта потеряла один из четырёх инструментальных рисков.

Значение этого результата выходит за саму строку реестра. Во-первых, закрыт последний методологический разрыв между двумя крыльями: любое число, которое

численное крыло готово подписать строгой оценкой, теперь может легально войти в теорию. Во-вторых, создан шаблон закрытия инструментальных пробелов: спецификация → доказательство → сертификатный принцип → регрессионная блокировка; этот шаблон уже применён к мосту и переиспользуется дальше. В-третьих, маршрут существования получил своего поставщика строгих оценок: интервально-ньютоновская полоса из главы 6 опирается именно на доказанные здесь гарантии.

3.7. Тесты интервального слоя

Доказанные теоремы и тесты в этом слое не дублируют, а дополняют друг друга. Теоремы говорят о конструкции типа Interval в Lean: каковы бы ни были входы, свойство покрытия выполнено. Тесты говорят о Python-реализации: на зафиксированных наборах входов DirectedRounder действительно расширяет результат наружу на каждом шаге, зависимость от платформы отсутствует, а граничные случаи — нули, отрицательные диапазоны, вырожденные отрезки — обрабатываются по спецификации. Вместе они закрывают обе половины вопроса: правильность конструкции и правильность её воплощения.

Таблица 6. Интервальный слой: что гарантируется чем

Гарантия	Носитель	Проверка
покрытие при сложении и умножении	теоремы add_sound, mul_sound в Lean	ядро доказателя, аудит аксиом
внешне-округлённый расчёт на сервисе	interval_checks на math.nextafter	pytest-регрессии, граничные случаи
переход оценки в теорию	конечная точка interval_bound	схемный контроль + regression-lock
звукость образа полосы	разрядка correction_band_image_sound	ядро, по рациональным границам сертификата

Строка таблицы, заслуживающая отдельного взгляда, — конечная точка interval_bound. Она соединяет два мира: клиент шлёт коэффициенты многочлена и отрезок домена, сервис отвечает внешне-округлённой оценкой образа. Ответ по построению корректен, но может быть шире точного образа — проблема зависимости из раздела 3.5; протокол честно документирует это в примечании о том, что звукость — свойство, на которое опирается доказательная сторона, а теснота — забота численной стороны. Такое документирование предостережений на границе — стиль программы: каждая слабость, которая не исправляется, обязана быть названной.

3.8. Почему не float-доказательства

Возможный вопрос: зачем вся эта конструкция, если можно формализовать саму арифметику с плавающей запятой и доказывать утверждения прямо о double'ах? Ответ — экономика доверия. Формализация IEEE-754 существует и осмысленна, но она тянет за собой режимы округления, денормали, особые значения и платформенные различия — пространство состояний, в котором каждая теорема обрастает техническими оговорками. Рациональный интерфейс с внешне-округлённым производителем отделяет случайность платформы от смысла утверждения: теория говорит о числах, а не о реализациях чисел.

Есть и второй ответ — институциональный. Доверенная база программы — ядро доказателя и три аксиомы; введение float-семантики в теорию добавило бы в эту базу целый стандарт. Рациональный канал добавляет ноль: рациональные числа уже в ядре, norm_num уже проверен сообществом mathlib, сертификаты — обычные данные. При прочих равных программа выбирает тот вариант, который ничего не добавляет в список того, во что нужно верить. Это правило общего назначения, и оно снова сыграет в главах 5 и 9.

4. Мост Lean⇄Python: типизированная граница

Между доказателем и численным сервисом лежит граница — и вся честность программы решается на этой границе. Мост устроен как контракт: Lean 4 — клиент, инициирующий каждый запрос; Python-сервис — сервер, отвечающий на запросы и никогда не выступающий инициатором; полезные нагрузки — JSON-объекты в UTF-8; числа — двойной точности всюду, где речь об интерактивной работе, и запрещены всюду, где речь о доказательстве. Настоящая глава разбирает контракт метод за методом, затем — клиентскую сторону и дисциплину тестов, которая не даёт контракту дрейфовать.

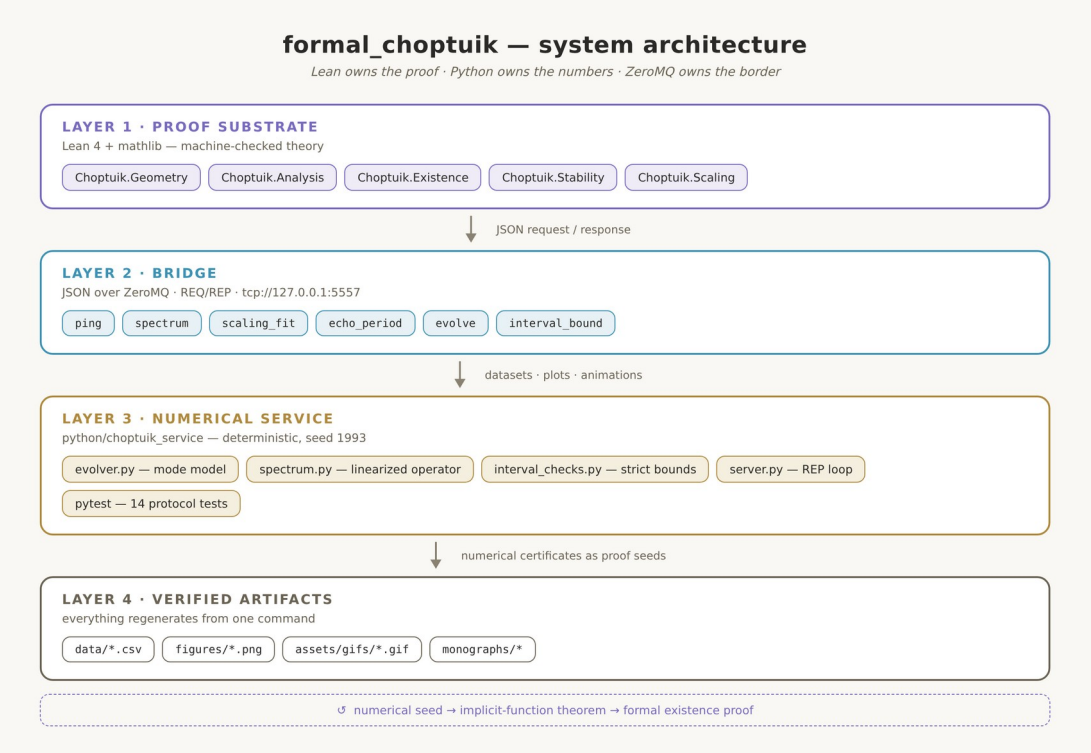


Рис. 3. Архитектура моста: интерактивный канал (справа) работает с числами с плавающей запятой для исследования и калибровки; доказательный канал (слева) переносит только точные рациональные сертификаты через коммитимые артефакты.

4.1. Два канала: интерактивный и доказательный

Каналов два, и их разделение — главное архитектурное решение моста. Интерактивный канал — это ZeroMQ-пара сокетов REQ/REP: Lean-клиент шлёт JSON-запрос на tcp://127.0.0.1:5557, сервис отвечает, клиент разбирает ответ схемно-контролируемым декодером с таймаутами, закрывающимися в отказ. Числа здесь плавающие, потому что назначение канала — исследование и калибровка: посмотреть спектр, подогнать показатель, проверить период эха, получить профили эволюции в запрошенные моменты логарифмического времени. Ни один результат интерактивного канала не входит в доказательство напрямую — это граница, о которой протокол напоминает явно.

Доказательный канал — офлайн и файловый. Производитель сертификатов вычисляет величины, округляет их наружу до точных десятичных рациональных, пишет JSON-артефакты и генерирует Lean-модуль с этими же числами. Артефакты коммитятся в репозиторий и становятся частью дерева: их проверяет ядро, их потребляют теоремы разрядки, их байт-стабильность заперта тестами. Разделение решает задачу, которую наивный мост решает плохо: как потреблять вычисления в доказательстве, не доверяя ни процессу вычисления, ни транспорту. Ответ: не потреблять процесс вовсе — потреблять проверенный результат.

4.2. Протокол: конверт, семь методов, коды ошибок

Протокол задаёт конверт запроса и ответа. Запрос — объект с полем method и параметрами; успешный ответ — объект со статусом ok и результатами; ошибка — статус error с кодом и сообщением. Коды ошибок перечислены исчерпывающе: неверный JSON, неизвестный метод, отсутствующее поле, некорректное значение поля, невосстановимый эхо-пик. Исчерпывающий перечень — не формальность: клиент обязан обрабатывать каждый код, и новый код не может появиться незаметно, потому что изменение протокола блокировано парным тестом.

Таблица 3. Методы протокола и их доказательные потребители

Метод	Возвращает	Потребитель на стороне Lean
ping	версия и калиброванные константы	санитарная проверка сеанса
spectrum	собственные значения, признак единственности неустойчивой моды	калибровочная цель G2
scaling_fit	подгонка γ и остаток	сверка с mass_scaling_law
echo_period	восстановленный период Δ	сверка с echoingPeriod = ln 2
evolve	профили в запрошенные моменты τ	численное зерно для маршрута G1
interval_bound	строгая внешне-округлённая оценка [lo, hi]	строгие границы для маршрута IFT
certificate	рациональный сертификатный пакет (4 вида)	Generated-модуль — доказательный канал

Седьмой метод — certificate — появился с каналом сертификатов и замкнул перечень. Его особенность в том, что ответ обязателен к точной рациональной форме: каждое число — точная десятичная строка, а не float; каждое поле — часть схемы с версией производителя и протокола. Метод выполняет двойную роль: интерактивно он отдаёт сертификат по запросу, а в офлайн-режиме те же артефакты лежат в bridge/certificates и попадают в Lean через сгенерированный модуль. Двойная роль не создаёт двух источников истины: артефакты байт-идентичны, и тесты сверяют оба пути.

4.3. Типизированный клиент

Клиентская сторона моста — это не «просто послать JSON». Типизированный клиент описывает схему каждого ответа: какие поля обязательны, каких типов, в каких диапазонах; декодер отклоняет ответ, не соответствующий схеме, вместо того чтобы протаскивать его дальше; таймауты закрываются в отказ — отсутствие ответа трактуется как ошибка, а не как пустой успех. Fuzz-тесты гоняют по декодеру

искажённые, обрезанные и злонамеренно устроенные payloads и требуют либо корректного разбора, либо честного отказа — никогда не исключения и не зависания.

На стороне Lean существует иллюстративный клиент, показывающий паттерн использования FFI для транспорта. Он намеренно не входит в теоретическую сборку и не несёт доказательного содержания: транспортная иллюстрация и доказательный канал — разные вещи, и программа держит их отдельно, чтобы иллюстративный код не создавал ложного впечатления проверенности. Реальный путь данных в доказательство проходит через артефакты и сгенерированный модуль, как описано в главе 5; интерактивный клиент — инструмент исследователя, а не канал доказательства.

4.4. Тестовая дисциплина

Шестьдесят пять тестов моста разбиты на три семейства по слоям. Сервисные тесты (семнадцать) запирают каждый метод протокола: схемы запросов и ответов, коды ошибок, детерминизм, физические регрессии — спектр с единственной неустойчивой модой, подгонку показателя, восстановление периода. Сертификатные тесты (восемнадцать) проверяют производителя: звукость сертификатов, точность рациональных представлений, байт-стабильность регенерации. Клиентские тесты (тридцать) закрывают декодер и транспорт: кодеки, fuzz, живой round-trip по ZMQ, таймауты. Три семейства соответствуют трём участникам контракта, и ни один из них не может измениться, не разбудив собственное семейство.

Регрессионная блокировка протокола — институциональная, а не техническая дисциплина: изменение protocol.md без парного теста в том же коммите запрещено конвенцией репозитория. Мост, таким образом, живёт по тем же правилам, что и теоремы: изменение контракта требует свидетельства, свидетельство требует коммита, коммит требует зелёной сборки. Эта преемственность дисциплины между кодом и доказательством — одна из отличительных черт программы: она устраняет ситуацию, в которой теория проверена, а её входные данные поменялись без ведома доказательства.

4.5. Два скрытых дефекта и чему они научили

История моста содержит и честный эпизод: при переводе протокола на версию 1.1 аудит вскрыл два латентных дефекта сервера первой версии — оба не проявлялись на штатных нагрузках и оба были бы неприятны на граничных. Дефекты были исправлены с парными тестами, воспроизводящими граничные ситуации, и вошли в перечень изменений протокола. Эпизод показатель вдвойне: он показывает, что даже дисциплинированный код несёт скрытые дефекты, и он показывает, как программа их обрабатывает — регистрация, исправление, парный тест, запись в журнале изменений.

Ни одно из этих действий не потребовало доверия к памяти разработчика: всё зафиксировано в дереве.

Урок для архитектуры доверия простой: граница между доказательством и вычислением должна быть проверяема на обоих своих берегах, и проверка одного берега не заменяет проверку другого. Lean-сторона проверяет утверждения о числах; Python-сторона проверяет схемы, коды и стабильность; только совокупность двух проверок делает мост тёмным местом с освещёнными обоими концами. Дальнейшая глава — о том, что происходит с сертификатом после того, как он перешёл границу.

4.6. Эволюция протокола: от шести методов к семи

Протокол моста — живой документ, и его история показательна. Первая ревизия описывала шесть методов: проверку живости, спектр, подгонку показателя, период эха, эволюцию и интервальную оценку. Канал сертификатов добавил седьмой метод — certificate — и заодно прошёл аудит всей границы: два латентных дефекта сервера были найдены и исправлены с парными тестами, схемы ответов уточнены, семейство клиентских тестов выросло до тридцати. Ревизия была проведена по правилу paired tests: ни одно изменение спецификации не вошло в дерево без теста, который его запирает.

Таблица 7. Слои тестирования моста

Семейство	Объём	Что запирает
сервисные тесты	17	схемы и коды каждого метода, детерминизм, физические регрессии: спектр, подгонка, период
сертификатные тесты	18	звукость и точность сертификатов, байт-стабильность регенерации, закрытость перечня видов
клиентские тесты	30	кодеки, fuzz-нагрузки, живой ZMQ round-trip, таймауты, отказоустойчивое поведение

Дисциплина paired tests имеет прямое отношение к архитектуре доверия. Контракт, который меняется без свидетельства, — это не контракт, а привычка; мост же является частью доказательной инфраструктуры, и потому его изменения идут по тем же правилам, что изменения теорем: свидетельство в том же коммите, зелёная сборка, запись в журнале. Практический эффект виден в истории: после перевода на седьмой метод ни один ответ сервера не изменил схемы незаметно для клиента — потому что незаметных изменений в этой системе не бывает.

4.7. Отказы и их семантика

Мост продуман с точки зрения того, что происходит, когда что-то идёт не так. Исчерпывающий перечень кодов ошибок делит отказы на классы: неверный payload, неизвестный метод, отсутствующее поле, некорректное значение, невосстановимый пик эха. Клиент обязан обрабатывать каждый класс явно: таймаут закрывается в отказ — отсутствие ответа никогда не трактуется как успех; нестандартный ответ отбрасывается декодером схем, а не протаскивается дальше; fuzz-нагрузки не приводят ни к исключениям, ни к зависаниям. В доказательном канале семантика отказов ещё проще: артефакт, не прошедший проверку, не попадает в теорию вовсе — сборка падает на сьюте, а не на теореме.

Такая строгость к отказам — продолжение принципа fail closed. Система, в которой отсутствие данных означает «продолжить без проверки», обречена однажды продолжить без проверки в неподходящий момент. Программа выбирает обратное значение по умолчанию: нет ответа — нет данных; нет проверки — нет теоремы; нет сертификата — нет разрядки. Все три формулировки — одна и та же позиция, применённая к транспорту, к проверке и к доказательству. Она делает поведение системы предсказуемым ровно там, где предсказуемость дороже всего — в отказах.

5. Канал сертификатов: точные рациональные от края до края

Канал сертификатов — сердце доказательной половины моста и главный технический сюжет настоящего тома. Его можно описать одной фразой: производитель вычисляет и округляет наружу, дерево хранит и распространяет, ядро проверяет, теоремы разряжают. За этой фразой стоит конвейер из пяти звеньев, и каждое звено устроено так, что отказ любого из них виден немедленно. Настоящая глава проходит конвейер от конца до конца: зачемfloat'ам запрещено пересекать границу, как устроен артефакт, какие виды сертификатов существуют, как сгенерированный модуль попадает в сборку и что делают теоремы разрядки.

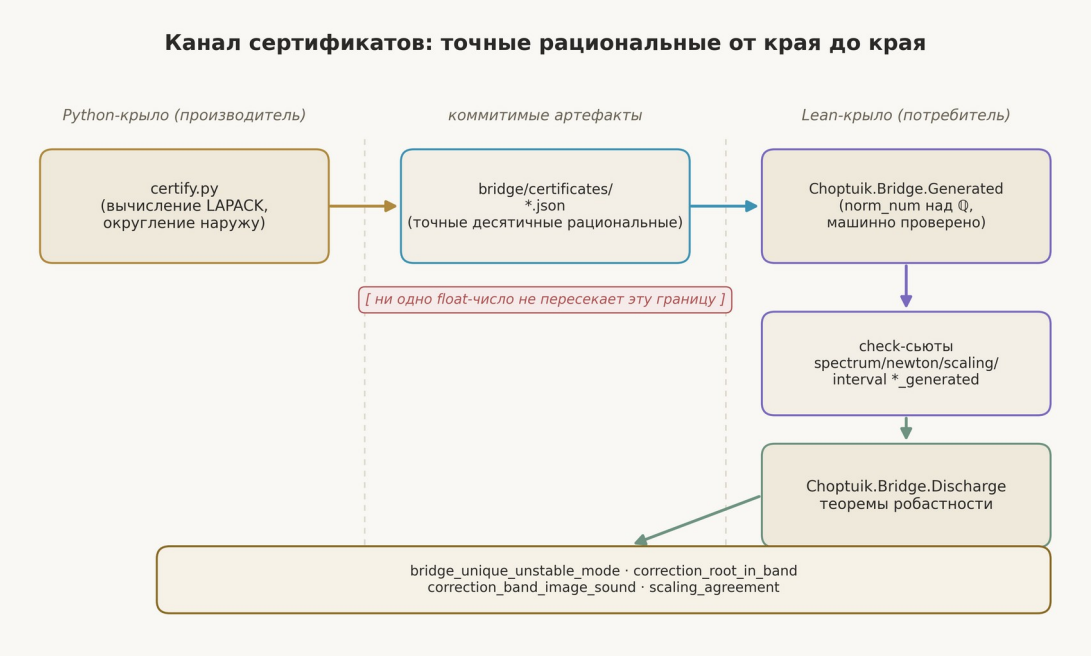


Рис. 4. Канал сертификатов от конца до конца: производитель округляет наружу до точных рациональных; коммитимые артефакты и сгенерированный Lean-модуль образуют доказательную половину; check-сьюты и теоремы разрядки превращают данные в итоговые теоремы.

5.1. Почему числа с плавающей запятой не пересекают границу

Число с плавающей запятой в доказательном слое — это обязательство, которое никто не подписывал. Ядро Lean не умеет проверять утверждения о double'ах напрямую: арифметика с плавающей запятой в формальной логике — это огромное пространство режимов округления, денормалей и платформенных особенностей, и тащить его в теорию значит умножать доверенную базу. Программа выбирает другой путь: на границе канала каждое число переводится в точное десятичное представление рационального числа — строку вида «2.6744999», — и уже эти строки становятся данными теории. Рациональное число в Lean — конструктивный объект с точными операциями; norm_num проверяет равенства над ним арифметически, до последнего знака.

Важно понимать, что перевод в рациональные не делает числа «более истинными» — он делает их проверяемыми. Округление наружу даёт отрезок, который гарантированно содержит вычисленное значение вместе со всей погрешностью вычисления; границы отрезка записываются как точные десятичные строки; утверждение «истинное значение лежит между этими границами» — это уже высказывание, которое ядро может проверить относительно данных. Погрешность не исчезла — она переехала внутрь ширины отрезка и стала частью проверяемого контракта. Так честность вычисления конвертируется в проверяемость доказательства без единого шага веры.

5.2. Анатомия сертификата

Сертификат — это JSON-документ со схемой, версией производителя и протокола, видом и телом. Схема фиксирует, какие поля обязательны и каких форм; версия производителя позволяет отличать регенерации; вид определяет интерпретацию тела. Все рациональные значения передаются точными десятичными строками; всё число — то есть и вычисленное значение, и границы его внешне-округлённой окрестности, и модельное эталонное значение для сверки — присутствует в явном виде. Регенерация сертификата байт-в-байт воспроизводима на любой корректной реализации LAPACK: детерминизм заперт отдельным семейством тестов, и коммит байтов, отличающихся от регенерированных, невозможен.

Байт-стабильность — не педантизм, а несущая конструкция доверия. Артефакт коммитится вместе с деревом, и теоремы ссылаются на его содержимое; если бы регенерация могла дрейфовать, каждая пересборка ставила бы под сомнение все ссылающиеся доказательства. Поскольку регенерация стабильна, артефакт ведёт себя как константа теории: его можно читать, проверять, цитировать — но не редактировать руками. Правило «не редактируй генерируемое» внесено в конвенцию репозитория наравне с запретом проектных аксиом.

5.3. Четыре вида сертификатов

Таблица 4. Виды сертификатов и их Lean-потребители

Вид	Содержание	Lean-путь
spectrum	рациональные окрестности вычисленного спектра усечённой модальной модели (семь пар затухающих мод)	check-сьют spectrum → разрядка bridge_unique_unstable_mode
newton	константы Ньютона–Канторовича (β, L, r, R) и интервально-ньютоновская полоса уравнения поправки	check-сьют newton → correction_root_exists, correction_root_in_band
scaling	окрестность подгонки глубокого масштабирования γ против $1/\lambda_0$	check-сьют scaling → scaling_agreement
interval	точная рациональная оценка образа полосы по схеме Горнера	check-сьют interval → correction_band_image_sound

Четыре вида закрывают ровно те величины, которые нужны маршрутам существования и стабильности. Спектральный сертификат несёт окрестности всех вычисленных собственных значений: по ним теория проверяет, что ровно одно значение лежит в правой полуплоскости, а остальные — в левой, и что мнимые части затухающих пар отстоят друг от друга на $2\pi/\Delta$ в пределах допуска. Ньютоновский

сертификат несёт четыре константы количественной теоремы о неявной функции и полосу для уравнения поправки. Масштабный — окрестность подгонки показателя против reciprocal первой моды. Интервальный — образ полосы, полученный честной внешне-округлённой схемой Горнера. Неизвестный вид — ошибка bad_input; перечень закрыт, и новый вид требует расширения схемы с парным тестом.

5.4. Generated-модуль: данные, проверенные ядром

Второй артефакт производителя — сгенерированный Lean-модуль, содержащий те же числа в виде рациональных литералов и четырёх check-сьютов. Каждый сьют — это список пар «утверждение о числах, доказательство» : утверждение — например, что нижняя граница окрестности собственного значения меньше модельного значения, а верхняя больше, — и доказательство строится механической процедурой нормализации рациональной арифметики. Ядро проверяет каждую пару; модуль собирается как обычная часть теории; данные с этого момента неотличимы от аксиоматически честной константы — с той разницей, что их происхождение видно и воспроизводимо.

Сгенерированный модуль — ответ программы на классический вопрос формализации: как ввести в доказательство вычислительные данные, не вводя аксиом и не вручную перепечатывая числа. Ответ: числа попадают в теорию как данные с механической проверкой их заявленных свойств; никакая часть вычисления не считается доверенной; и если завтра данные изменятся, пересборка немедленно покажет расхождение со ссьютами. Ручное редактирование исключено конвенцией, а не честным словом: файл перезаписывается производителем целиком.

5.5. Check-сьюты и теоремы робастности

Между «данные проверены» и «физика следует» лежит слой теорем робастности: высказываний вида «если проверки сертификата выполнены, то физическое утверждение верно». Например: если проверенный спектр показывает ровно одно собственное значение в правой полуплоскости с рациональной окрестностью, не содержащей нуля, то линеаризованный оператор модели имеет единственную неустойчивую моду. Или: если проверенные константы удовлетворяют неравенствам Ньютона–Канторовича, то в полосе существует корень уравнения поправки, и итерация с заданного носителя в него сходится. Эти теоремы — мост между миром данных и миром утверждений; их формулировки фиксированы, их доказательства проверены ядром, и их гипотезы заперты на данных.

Робастная форма — принципиальное методологическое решение. Вместо того чтобы формулировать спектральную теорему с абсолютными константами, программа формулирует её условно относительно проверяемого сертификата, и

условие проверяется машиной. Когда в будущем появится сертификат для истинной системы Эйнштейна–скаляра, та же самая теорема робастности применится к новым данным без единого изменения своего доказательства. Именно так инфраструктура, построенная сегодня, сокращает путь к доказательствам завтрашнего дня: физика обновляется заменой данных, а не переписыванием теории.

5.6. Разрядка в итоговые теоремы

Финальное звено конвейера — разрядка: вывод безусловных теорем о модели из теорем робастности, применённых к проверенным сертификатам. Четыре итоговые разрядки сопровождают четыре вида сертификатов: спектральная разрядка даёт единственность неустойчивой моды модели; ньютоновская — существование корня уравнения поправки в полосе; масштабная — согласие подгонки с reciprocal первой моды в заданном допуске; интервальная — звукость образа полосы. Каждая разрядка печатает свои аксиомы и входит в тридцать три итоговые теоремы аудита. Таким образом, канал сертификатов уже сегодня доставляет в теорию настоящие, проверенные, безусловные результаты о модели — и готов доставлять те же результаты об истинной системе, как только её сертификаты будут произведены.

Собирая главу, зафиксируем инвариант канала: на каждом звене конвейера данные либо проверены, либо не попали в теорию. Производитель подписывает оценки шириной; артефакты хранят байты, которые нельзя незаметно изменить; сгенерированный модуль проходит ядро; теоремы робастности переносят проверенные свойства в физические утверждения; разрядка делает их безусловными. Доверенная база остаётся прежней — ядро и три аксиомы; всё остальное — проверка.

5.7. Поля артефакта: что именно подписывает производитель

Рассмотрим артефакт глазами проверяющей стороны. Шапка несёт схему, вид, производителя с его версией и версию протокола — по ней проверяющая сторона знает, какой интерпретации ожидать и каким набором сьютов проверять. Тело — видозависимо: для спектрального это окрестности всех вычисленных собственных значений вместе с модельными эталонами; для ньютоновского — четыре константы и полоса; для масштабного — окрестность подгонки и reciprocal первой моды; для интервального — рациональные границы образа. Все числа — точные десятичные строки; никакой плавающей точки в файле нет и быть не может — формат запрещает её грамматически.

Таблица 8. Инварианты сертификатного артефакта

Инвариант	Кто обеспечивает	Кто проверяет
все рациональные — точные десятичные строки	формат схемы	парсер сьютов в Lean
окрестности содержат вычисленные значения	внешне-округлённый расчёт производителя	pytest-регрессии soundness
регенерация байт-в-байт	детерминированный конвейер, seed	сертификатные тесты, CI
перечень видов закрыт	диспетчер производителя	bad_input на неизвестный вид
проверяемость заявленных неравенств	нормализация рациональной арифметики	ядро Lean при сборке

Ключевой инвариант — последний: заявленные неравенства проверяемы ядром. Это значит, что производитель не может «пообещать» что-то непроверяемое — любая заявка либо нормализуется до точного равенства, либо отклоняется сборкой. Отсюда следует дисциплина производства: константы, которые не удаётся обосновать рациональной оценкой, не попадают в сертификат, а идут в интерактивный канал как исследовательский материал. Граница между этими мирами проходит ровно по линии проверяемости, и она неподвижна.

5.8. Жизненный цикл сертификата

У сертификата есть полный жизненный цикл, и все его стадии автоматизированы. Рождение: производитель вычисляет, округляет наружу и пишет артефакты. Взросление: сгенерированный модуль попадает в сборку Lean, сьюты проверяются ядром, теоремы робастности применяются, разрядки пополняют аудит. Зрелость: артефакт цитируется теоремами, его байты заморожены регенерационными тестами. Старение: если данные изменяются — новая калибровка, новая модель, новый оператор, — старые артефакты перегенерируются, и вся цепочка от сьютов до разрядок переезжает на новые числа без изменения доказательств. На каждой стадии состояние проверяемо; ручных переходов нет.

Жизненный цикл разъясняет и политику наследования. Теорема робастности, однажды доказанная, не знает, чей сертификат к ней применён, — она знает лишь схему проверок. Поэтому смена поколения данных не требует смены поколения теории, и наоборот: теория может развиваться (новые леммы, tighter разрядки), не трогая данные. Развязка поколений — то, что позволяет программе двигаться одновременно в двух темпах: быстрым темпом вычислений и медленным, торжественным темпом доказательств — не связывая их жёстко и не позволяя одному тянуть другое вниз.

6. Путь к существованию критического решения: гэп G1

Существование критического решения — центральный блокер программы: пока оно не доказано, остальной сюжет стоит на песке. Математический объект, о котором идёт речь, — гладкое дискретно-самоподобное решение сферически симметричной системы Эйнштейна–скаляра, к которому стягиваются эволюции на пороге образования чёрной дыры. Компьютерно-ассистированное доказательство существования выполнили Райтерер и Трубовиц; задача программы — превратить его в доказательство, проверенное ядром. Настоящая глава излагает маршрут: систему поля, определение самоподобности, стратегию коррекции Ньютона и то, что из этой стратегии уже доказано в Lean.

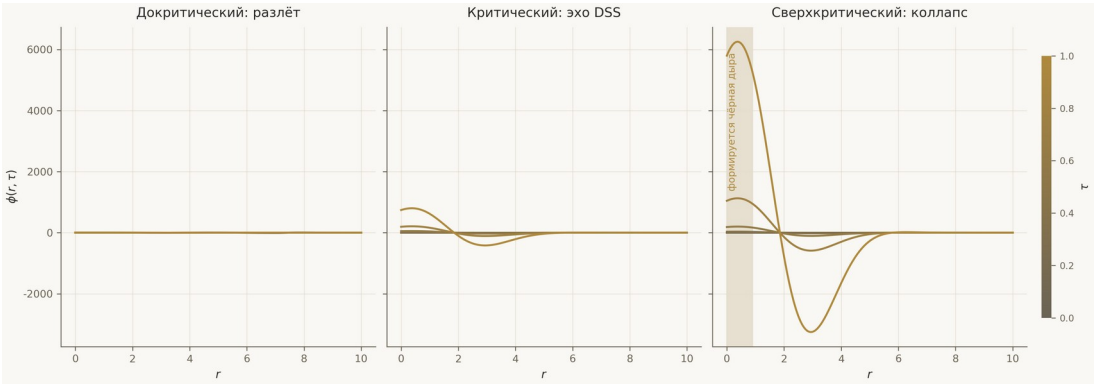


Рис. 5. Профили коллапса: докритические эволюции рассеиваются, критическая стягивается к самоподобному решению, суперкритические образуют чёрные дыры; порог между ветвями и есть критическое решение.

6.1. Система Эйнштейна–скаляра в двойных нулевых координатах

Материальное содержание задачи — безмассовое скалярное поле, минимально связанное с гравитацией: уравнения Эйнштейна с тензором энергии-импульса скалярного поля плюс волновое уравнение на метрике, которую поле само же и источает. В сферической симметрии стандартный выбор — двойные нулевые координаты (u, v), в которых метрика приводится к виду с двумя функциями: конформным фактором и радиусом площади. Система сводится к связанной системе первого порядка с нулевыми ограничениями вдоль характеристик; корректность характеристической задачи — первый содержательный аналитический факт, который должна доставить формализация.

Аспект массы — величина, чей ненулевой предел на будущей нулевой бесконечности удостоверяет образование чёрной дыры. Семейство начальных данных параметризуется амплитудой p; критическая амплитуда p* определяется как граница между ветвью рассеяния и ветвью коллапса. Для формализации важно, что p* не постулируется, а наследуется из теории бифуркаций: существование и гладкость

границы — часть того, что в конечном счёте даёт теорема существования и теория устойчивости. Модуль Analysis/HyperbolicPDE фиксирует систему и ограничения; выход за каркас — задача задачи P2.1 второго этапа.

6.2. Дискретная самоподобность и закон ковариации эха

Критическое решение не стационарно и не непрерывно самоподобно — оно дискретно самоподобно. В логарифмическом времени τ = −ln(T − t) решение воспроизводит себя при сдвиге на период Δ вместе с пересчётом радиуса: поля преобразуются по закону ковариации с показателем κ. Численно Δ ≈ ln 2: за один период эха все характеристические радиусы уменьшаются вдвое. Это определение воспроизведено в модуле Existence/DSS как предикат над функциями двух аргументов, и вокруг него построена вся бухгалтерия самоподобности: определение фундаментальной области, репараметризационная инвариантность, единственность с точностью до масштаба и сдвига.

$$\psi(\rho,\tau+\Delta)=e^{\{\kappa\Delta\}}\cdot\psi(\rho\,e^{\{-\Delta\}},\tau),\Delta=\ln 2$$

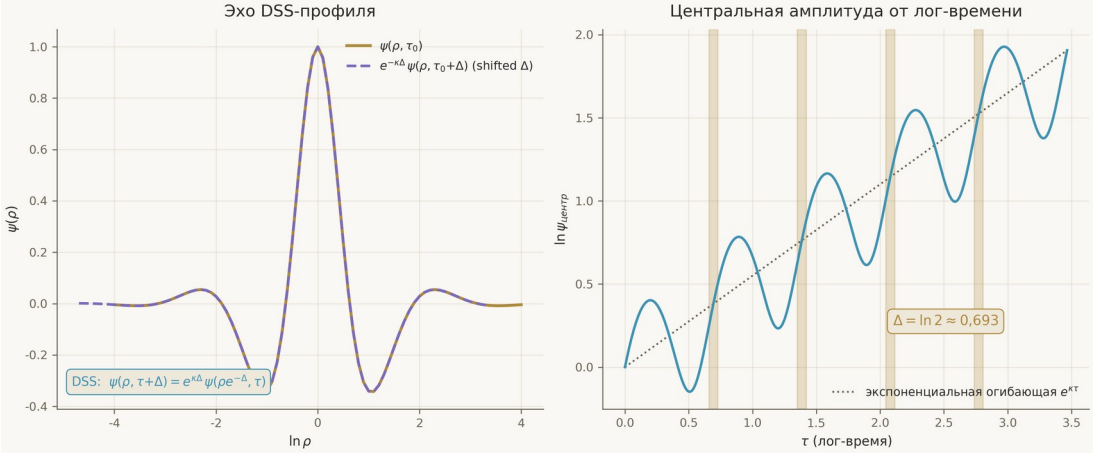


Рис. 6. Эхо ДСС: профиль, пересчитанный с радиусом, уменьшенным в e^{Δ} раз, и сдвинутый по τ на Δ, совпадает с профилем периода тому назад; экспоненциальная огибающая несёт периодическую модуляцию.

Дискретная самоподобность — не украшение, а несущая конструкция доказательства существования: именно она сводит бесконечномерную задачу к задаче на фундаментальной области с согласованием на концах. В Lean это выражено структурой модуля: предикат самоподобности, лемма об определении фундаментальной области, лемма о репараметризационной инвариантности и теорема единственности образуют каркас, на который в дальнейшем лягут ньютоновские оценки. Бухгалтерия ДСС — то место, где формализация освобождает физику от многозначных слов: «решение повторяет себя с периодом Δ» превращается в высказывание с кванторами, которое можно доказывать и опровергать.

6.3. Стратегия Райтерера–Трубовица: профиль плюс Ньютон

Стратегия доказательства — классическая для задач с вычислительным профилем: взять численно вычисленный профиль за нулевое приближение, построить в банаховом пространстве экспоненциально затухающих последовательностей оператор невязки, линеаризовать, доказать компактность обратного к линеаризации и применить количественную теорему о неявной функции. Райтерер и Трубовиц провели этот план с целочисленными оценками ошибок; программа переносит его в Lean по частям, и перенос устроен так, что каждый кусок проверяем независимо: пространство и оператор — в модуле Existence/ImplicitFunction; бухгалтерия самоподобности — в Existence/DSS; количественное ньютоновское ядро — там же; связь с численным профилем — через канал сертификатов.

Выбор банахова пространства — экспоненциально затухающие последовательности — диктуется спектральной структурой задачи: затухание гасит вклады устойчивых мод и оставляет конечномерную неустойчивую часть, которой управляет амплитуда. Компактность обратного к линеаризованному оператору — самая тяжёлая аналитическая часть; в дорожной карте она помечена как главный риск второго этапа, и заготовленный запасной путь — формализовать опубликованные неравенства как самостоятельную главу операторной теории, не опираясь на общую спектральную теорию mathlib. Количественная форма теоремы о неявной функции нужна потому, что оценки сертификатов — численные: мало «существует корень рядом», нужно «корень лежит в полосе вот с этими рациональными границами».

6.4. Ньютоновско-канторовичевское ядро в Lean

Количественное ядро маршрута — ньютоновско-канторовичевская теорема в форме, пригодной для сертификатов: если отображение C^1, производная в шаре обратима с константой L, вторая производная ограничена константой β, носитель лежит в шаре радиуса r, и произведение 2βrL меньше единицы, то ньютоновская итерация с носителя сходится к единственному в шаре корню. В Lean доказаны и сжимаемость ньютоновского отображения, и единственность нуля в шаре, и существование через теорему о неявной функции, и устойчивость существования к возмущениям носителя — полный комплект, на который ложится сертификат newton с его четырьмя константами.

$$q := 2\beta r L < 1 \Rightarrow \| x_1 - x* \| \le \frac{\beta L}{2(1-q)} \| x_1 - x_0 \|^2, \; x* \text{ — единственный корень в шаре } B(x_0, r)$$

Сертификат newton подставляет в эту схему конкретные числа: константы β, L, r, R вычислены численно для модальной модели, округлены наружу и проверены сьютами; интервально-ньютоновская полоса удостоверяет, что образ носителя в

уравнении поправки лежит в нужных границах. Разрядка correction_root_in_band соединяет проверенные константы с доказанной теоремой и даёт безусловное утверждение о существовании корня в полосе для модели. Это не существование критического решения — это демонстрация всего аналитического контура на модели, и ценность её именно в том, что контур замкнут: от численных констант до безусловной теоремы, без единого неявного шага.

6.5. Доказанное и вынесенное в гипотезы

Соберём баланс маршрута. Доказано в Lean: бухгалтерия самоподобности целиком — ковариация, фундаментальная область, репараметризационная инвариантность, единственность с точностью до симметрий; каркас существования на уровне носителя и уникальности; количественное ньютоновско-канторовичевское ядро; устойчивость существования к возмущениям; локальная корректность системы поля в слабой форме. Вынесено в явные гипотезы: сертификат Райтерера–Трубовица для истинной карты невязки (его формат, проверяющая сторона и демонстрация на модели уже живы в канале); компактный обратный для истинного линеаризованного оператора. Гипотезы перечислены в реестре пробелов, видны в формулировках теорем и ждут сертификатов, а не инфраструктуры.

Такая архитектура маршрута даёт программе два свойства, которые редко встречаются вместе. Прогресс теперь аддитивен: каждый следующий кусок аналитики — компактность, оценки невязки, перенос на истинную систему — добавляется к готовому каркасу, не переписывая его. И обратная совместимость гарантирована: когда сертификат истинной системы прибудет по каналу из главы 5, вся цепочка от сьютов до разрядки применится без изменений. Существование критического решения перестаёт быть монолитом и становится конвейером — тем самым, который уже отлажен на модели.

6.6. Как G5 обслуживает G1: интервально-ньютоновская полоса

Связка между главами этой книги имеет точное техническое выражение. Интервально-ньютоновский шаг маршрута существования требует: для уравнения поправки на носителе вычислить внешний образ оператора и удостовериться, что нуль исключён из образа вне полосы, а внутри полосы выполнены условия сжимаемости. Точный образ — работа интервальной арифметики из главы 3: внешне-округлённая схема Горнера даёт рациональные границы образа, сертификат interval несёт их в теорию, разрядка correction_band_image_sound удостоверяет звукость образа, и полоса становится проверенным объектом, с которым работает ньютоновская теорема. Без закрытого G5 этот шаг оставался бы устной договорённостью; теперь он — конвейер.

Точно так же мост из главы 4 обслуживает маршрут численным зерном: конечная точка evolve отдаёт профили эволюции в запрошенные моменты логарифмического времени, из которых строится нулевое приближение ньютоновской итерации. Интерактивная поставка — для исследования; доказательная поставка того же материала — через сертификат. Оба пути дают одинаковые байты, и это совпадение тоже заперто тестами. Маршрут существования, таким образом, уже сейчас окружён полной обслугой — от зерна до разрядки; осталось произвести сертификат истинной системы, и это содержание второго этапа программы.

6.7. Инвентаризация аналитических компонентов маршрута

Маршрут существования состоит из аналитических компонентов разной природы, и их инвентаризация полезна для оценки оставшейся работы. В таблице ниже каждый компонент снабжён статусом и указанием, что именно осталось. Обратите внимание на асимметрию: всё, что относится к конструированию пространства, операторов и количественных оценок, уже проверено; всё, что осталось, относится к производству данных для истинной системы — то есть к численной, а не доказательной стороне.

Таблица 9. Компоненты маршрута существования и их статусы

Компонент	Статус	Осталось
предикат ДСС, фундаментальная область, репараметризация	доказано в Lean	—
единственность с точностью до масштаба и сдвига	доказано в Lean	—
локальная корректность системы поля (слабая форма)	доказано в Lean	полная форма — задача P2.1
ньютоновско-канторовичево ядро (количественное)	доказано в Lean	—
устойчивость существования к возмущениям	доказано в Lean	—
сертификат Райтерера–Трубовица для истинной невязки	формат и проверяющая сторона готовы	производство численных данных
компакный обратный для истинного оператора	стратегия зафиксирована	формализация, запасной путь готов

Читатель, знакомый с томом II, увидит здесь тот же список, что и в разборе центрального блокера, — но с другим акцентом: там список был планом, здесь он стал балансом. Пять строк закрыты доказательствами, две — инженерно подготовлены. Оценка усилий сместилась: главная неопределённость больше не в том, можно ли

формализовать стратегию, а в том, когда будут произведены её численные входы. Для исследовательской программы это качественный сдвиг — от «если» к «когда».

6.8. Нулевое приближение: откуда берётся профиль

Ньютоновской итерации нужен стартовый носитель, и его происхождение — часть доказательной гигиены. В модели нулевое приближение строится из профилей эволюции, которые сервис отдаёт конечной точкой evolve в запрошенные моменты логарифмического времени; численный профиль сглаживается в элемент банахова пространства, и итерация стартует. Для истинной системы тем же зерном послужит профиль из высокоточного солвера программы «Einstein Direct» — но попасть в теорию он сможет только привычным путём: сертификат с рациональными окрестностями ключевых величин профиля, сьюты, полоса для уравнения поправки.

Это ограничение — не бюрократия, а защита от тонкого источника ошибок. Профиль, принятый в теорию «как есть», приносит с собой все погрешности своего производства — дискретизацию, интегрирование, интерполяцию; ни одна из них не видна на глаз, и все они становятся невидимым фундаментом доказательства. Профиль, принятый через сертификат, приносит только то, что заявлено: рациональные границы, в которых лежат его характеристики. Ньютоновская теорема не требует точного нулевого приближения — она требует честной окрестности, и сертификатный формат даёт ровно это.

7. Путь к стабильности: спектр, масштабирование, универсальность (G2)

Если существование отвечает на вопрос «есть ли критическое решение», то стабильность отвечает на вопрос «почему его видят все». Линеаризованная задача на критическом решении должна иметь ровно одну неустойчивую моду: тогда фазовое пространство делится на критическую поверхность коразмерности один, и всякая траектория вблизи порога сначала прилипает к критическому решению, а потом уходит от него вдоль единственного неустойчивого направления — с показателем, который и есть наблюдаемый γ. Настоящая глава излагает спектральный маршрут: линеаризацию, ядро спектральной теории, вывод закона масштабирования, формулу эха и универсальность.

7.1. Линеаризация и оператор возмущений

Линеаризация системы Эйнштейна–скаляра на критическом решении даёт линейный оператор возмущений в логарифмическом времени. В силу самоподобности критического решения оператор коммутирует со сдвигами на период

эха, и его спектральное разложение естественно ведёт по модам с определённым поведением при сдвиге: неустойчивая мода — вещественная, растущая с показателем λ_0; устойчивые — пары с отрицательной вещественной частью и мнимыми частями, отстоящими друг от друга на $2\pi/\Delta$. Эта структура — мост между двумя независимыми фактами: периодичностью критического решения и дискретностью спектра. Формализация фиксирует интерфейс оператора в модуле Stability/Linearization; его выход за каркас — задача P3.1 третьего этапа.

В модальной модели оператор — блочная матрица конечного усечения, и задача на собственные значения решается честно: действительная задача 15×15, а не таблица констант. Вычисленный спектр показывает ровно одну моду в правой полуплоскости, семейство затухающих пар и максимальное отклонение межэховых расстояний от $2\pi/\Delta$ в пределах $2.0 \cdot 10^{-10}$. Модельный спектр — это тренировочный объект для всего конвейера разрядки: он достаточно сложен, чтобы проверка была содержательной, и достаточно прост, чтобы сертификат был компактным.



Рис. 7. Спектр линеаризованного оператора в модельной задаче: единственная неустойчивая мода $\lambda_0 = 2.6745$ (звезда), затухающие семейства в левой полуплоскости, межэховое расстояние $2\pi/\Delta \approx 9.06$.

7.2. Спектральное ядро: единственная неустойчивая мода

Спектральное ядро маршрута — теорема unique_unstable_mode в робастной форме: если проверенный сертификат спектра показывает ровно одно собственное значение в правой полуплоскости и все прочие — в левой, то оператор имеет единственную неустойчивую моду. Условная форма здесь не уступка, а метод:

абсолютная спектральная теорема для истинного оператора требует спектральной теории неограниченных операторов, самой тяжёлой зависимости третьего этапа, и дорожная карта сознательно скоупит её на конкретный оператор, а не на общую теорию. Проверяемая гипотеза отделяет математический контур от вычислительного содержимого — ровно тот приём, который отработан в предыдущих главах.

Разрядка bridge_unique_unstable_mode применяет спектральное ядро к проверенному сертификату и даёт безусловное утверждение о модели: единственность неустойчивой моды доказана, ядро печатает три стандартные аксиомы. Сертификатная цепочка при этом полностью видима: рациональные окрестности собственных значений, сьюты, теорема робастности, разрядка. Всякий, кто хочет оспорить результат, оспаривает не «доверие к коду», а конкретные рациональные границы — и для этого ему нужно лишь пересчитать спектр. Так спор о физике превращается в спор о числах, а спор о числах решается машиной.

7.3. От спектра к масштабированию: вывод $\gamma = 1/\lambda_0$

Вывод закона масштабирования — двухшаговый, и оба шага формализованы. Первый шаг: вблизи порога эволюция управляется растущей модой — амплитуда возмущения ведёт себя как |p − p*| · e^{λ_0τ}, и коллапс завершается, когда растущая мода достигает фиксированного размера; отсюда τ_max связан с ln|p − p*| линейно с коэффициентом $-1/\lambda_0$. Второй шаг: масса образовавшейся чёрной дыры экспоненциально мала по τ_max, откуда $M \propto |p - p^*|^{1/\lambda_0}$. В Lean доказана лог-лог форма закона: логарифм массы как функция логарифма расстояния до критической поверхности — линейная с наклоном $1/\lambda_0$ и периодическим украшением периода Δ — следствием дискретной самоподобности.

$$\ln M = \frac{1}{\lambda_0} \cdot \ln \vee p - p* ¿ + F\left(\ln \vee p - p* ¿ / \lambda_0 \right) + C \text{, период } F \text{ равен } \Delta$$

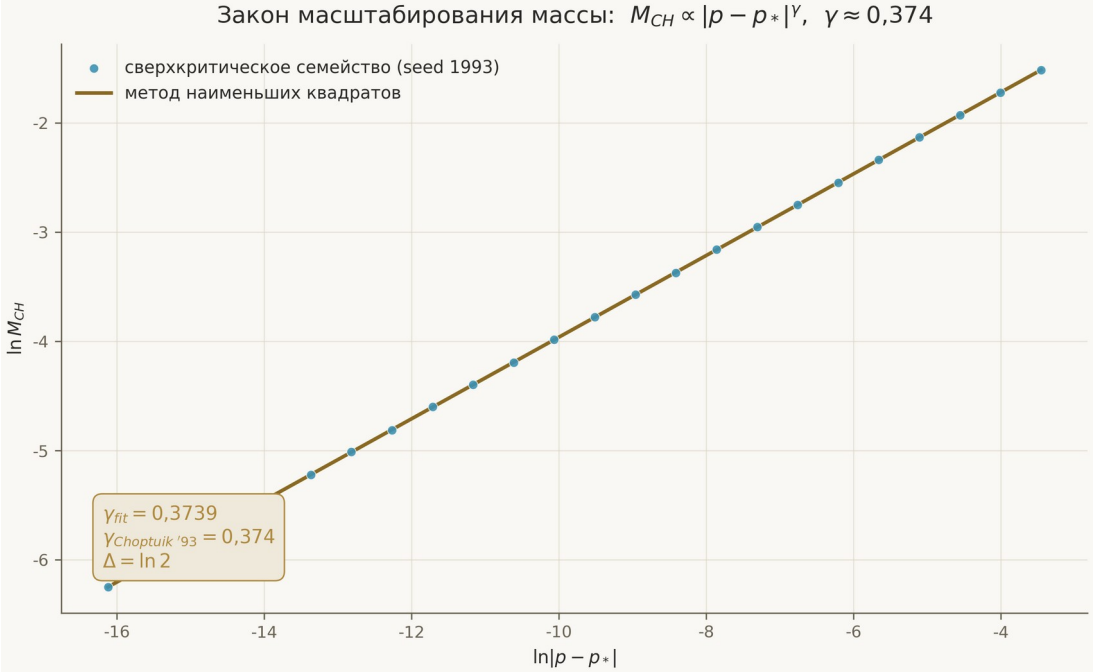


Рис. 8. Закон масштабирования массы по детерминированному суперкритическому семейству (seed 1993): наклон подгонки 0.3739 согласуется с опубликованным значением 0.374; периодическое украшение по оси ln|p−p| — след периода эха.*

Периодическое украшение — тонкая часть закона, которую численные работы наблюдали как «виляние» массы на логарифмической оси, а формализация получает бесплатно: сдвиг в ln|p−p*| на Δ/λ_0 эквивалентен сдвигу логарифмического времени на Δ, и самоподобность оставляет решение инвариантным. В Lean это оформлено как формула эха — теорема о структуре семейства эхо-сдвинутых решений, — и как следствие в лог-лог законе. Показатель $\gamma = 1/\lambda_0$ при этом не подгоняется, а выводится: сверка с численной подгонкой — отдельная теорема gamma_close_to_published, которая проверяет, что выведенный показатель совпадает с калиброванным до допуска, зафиксированного критерием приёмки.

7.4. Согласие доказательства и чисел

Отдельная разрядка — scaling_agreement — соединяет два независимых измерения одного показателя: выведенный из спектральной теории $1/\lambda_0$ и подогнанный по семейству суперкритических эволюций γ_fit. Сертификат scaling несёт рациональную окрестность подгонки; сьют проверяет, что reciprocal первой моды лежит в этой окрестности с допуском, согласующим доказательную и численную стороны; разрядка делает согласие безусловным утверждением о модели. Это тот самый момент, о котором говорят «доказательство и численность проверяют друг друга»: машинно проверенная теория предсказывает число, численный эксперимент его воспроизводит, а машинная проверка удостоверяет совпадение.

Согласие имеет и обратную сторону — оно ловит рассинхронизации. Если численное крыло изменит калибровку, подгонку или семейство, сьют масштабного сертификата немедленно заострится против теории; если теория изменит вывод показателя, тот же сьют заострится против чисел. Никакой третьей стороны, которая могла бы тихо пережить рассинхрон: инвариант «доказательство и числа согласны» поддерживается инфраструктурой, а не бдительностью. Именно в этом программа видит практический смысл машинной верификации — не в церемонии, а в автоматическом обнаружении расхождений.

7.5. Формула эха и универсальность

Формула эха — теорема о структуре критической эволюции под сдвигами: сдвинутая на Δ эволюция совпадает с пересчитанной по ковариационному закону; фундаментальная область — Δ-полоса логарифмического времени; все наблюдаемые величины квантуются по сдвигам этой полосы. В Lean она доказана в модуле Scaling вместе с универсальностью — утверждением, что семейства начальных данных разного происхождения видят один и тот же критический показатель и тот же период, поскольку оба определяются спектром одного и того же оператора. Универсальность формулируется условно относительно общности спектрального ядра — ровно как принято в программе.

Универсальность — внешне наблюдаемое предсказание всей конструкции: она объясняет, почему разные исследователи, разные сетки, разные семейства данных получали один и тот же γ. В формальной форме она же задаёт критерий успеха третьего этапа: если универсальность доказана безусловными теоремами, а γ выведен с допуском к опубликованному значению, то программа выполнила своё обещание — превратила численный фольклор критических явлений в теорему. До этого остался один шаг содержательной физики: спектральный сертификат истинного оператора.

7.6. Что осталось до G2

Баланс маршрута стабильности повторяет баланс маршрута существования: контур готов, содержимое — сертификат истинного оператора — в производстве. Доказано: спектральное ядро в робастной форме, разрядка на модельном сертификате, лог-лог закон масштабирования, согласие показателя с численной подгонкой, формула эха, условная универсальность. Вынесено в гипотезы: спаривание модельного спектра с истинным линеаризованным оператором системы Эйнштейна–скаляра. Задача P3.2 — произвести это спаривание: точное определение оператора на конусе затухающих последовательностей, оценки его спектра, сертификат. Дорожная карта отводит на это девятнадцать–двадцать третий месяцы; инфраструктура канала к

тому времени будет ждать данные уже готовой — как и случилось с остальными маршрутами.

7.7. Инвентаризация спектрального маршрута

Спектральный маршрут, как и маршрут существования, раскладывается на компоненты с независимыми статусами. Его особенность — больший вес вычислительной стороны: спектральная теорема об истинном операторе потребует не только аналитики, но и сертификата высокого качества, потому что рациональные окрестности собственных значений должны отделить единственную растущую моду от всего устойчивого семейства с запасом, который выдержит проверку неравенств. Инвентаризация ниже показывает, что конвейерной части маршрута почти нет — она построена; вся оставшаяся работа — анализ и вычисление для истинной системы.

Таблица 10. Компоненты спектрального маршрута и их статусы

Компонент	Статус	Осталось
интерфейс линеаризованного оператора	зафиксирован в Lean	полное определение — P3.1
спектральное ядро (робастная форма)	доказано в Lean	—
разрядка на модельном сертификате	доказано в Lean	—
лог-лог закон масштабирования	доказано в Lean	—
согласие показателя с подгонкой (допуск)	доказано в Lean	—
формула эха, условная универсальность	доказано в Lean	безусловная форма после G2
спаривание с истинным оператором	стратегия зафиксирована	определение оператора на конусе, оценки, сертификат

Допуск в строке согласия — тысяча-я часть published значения по критерию приёмки третьего этапа — заслуживает комментария. Допуск не делает доказательство «неточным»; он определяет, насколько тесной должна быть проверенная окрестность подгонки, чтобы согласие считалось установленным. Это то же отношение теории к числам, что и раньше: теория не подгоняет числа, она задаёт условие, при котором числа признаются согласованными, и проверяет его. Ужесточить допуск можно в любой момент — ценой более тесных сертификатов, а не переделки теории.

7.8. Эхо как организующий принцип спектра

Связь между периодом эха и структурой спектра — самая красивая часть сюжета, и формализация позволяет сказать о ней точнее, чем словесные описания. Сдвиг логарифмического времени на Δ переводит критическое решение в себя с пересчётом масштаба; следовательно, оператор возмущений коммутирует с этим сдвигом; следовательно, его спектральные подпространства организованы по собственным значениям оператора сдвига — а у сдвига на Δ они квантованы с шагом $2\pi/\Delta$ по мнимой оси. Дискретность межэховых расстояний — не наблюдение, а теорема; численное совпадение с $2\pi/\Delta$ до 10^{-10} — проверка этой теоремы на данных.

В обратную сторону та же логика работает как контроль качества данных: если бы вычисленный спектр показал межэховые расстояния, дрейфующие от $2\pi/\Delta$, это указывало бы не на «плохую выборку», а на нарушение самоподобности — и должно было бы остановить принятие сертификата. Ни один из таких остановочных механизмов не требует человеческого суждения: все они — следствия доказанных связей и проверяемых неравенств. Красота здесь совпадает с пользой: периодичность, которая делает критическое решение загадочным, делает его спектр проверяемым.

8. Геометрия сингулярности: гэп G3

Критическое решение кончается сингулярностью со сложной геометрией, и её формализация — третий содержательный сюжет программы. Два свойства определяют сцену: сингулярность сильная — интеграл кривизны вдоль нулевых геодезических расходится, инварианты типа Кречмана растут безгранично; и изотропная — компоненты Вейля остаются ограниченными, а конец пространства допускает конформную компактификацию. Комбинация «сила плюс ограниченный Вейль» отличает геометрию Чоптуика от шварцшильдовской и потому заслуживает собственной строки в реестре — G3, высокой серьёзности, разбитой между вторым и третьим этапами.

8.1. Сильная и изотропная: условное ядро

В Lean доказано условное ядро: если сертификаты коэрцитивности и вейлевских оценок выполнены, то сингулярность сильная и изотропная. Критерий силы — расходимость интеграла по нулевой геодезической; критерий изотропности — ограниченность вейлевских компонент в ковариационных масштабах. Оба критерия сформулированы как высказывания о геометрических объектах, а не как вычислительные рецепты, и теорема соединяет их с сертификатами через робастную форму — общий приём программы. Условная форма здесь особенно естественна: производитель сертификатов для геометрии — это будущая геометрическая ветвь численного крыла, и его отсутствие не должно блокировать аналитический контур.

Второй доказанный фрагмент — инвариантность расходимости при репараметризации: критерий силы не зависит от выбора представителя в классе самоподобия, что необходимо для корректности самой постановки. Без этой леммы критерий силы был бы привязан к произволу координат, и его формулировка не имела бы геометрического смысла. Оба результата живут в модуле Geometry/Curvature и замыкают на себя каркас Geometry/Basic — арену двойных нулевых координат, положительность радиуса площади и аспект массы.

8.2. Слой конформной компактификации: что потребуется

Незакрытая половина G3 — конформная компактификация конца пространства у горизонта Коши. Слой потребует расширения дифференциально-геометрического слоя mathlib: определения конформно компактной метрики, критерия расширимости, связки с нулевой геодезической структурой на границе. Это та же категория работы, что и компакный обратный в маршруте существования: библиотека ещё не содержит готового инструмента, и программа либо дособерёт его как самостоятельную главу, либо формализует минимальный скоуп, достаточный для утверждения о компактифицируемости именно чоптуиковского конца. Оба пути расписаны в дорожной карте; выбор между ними — вопрос второго-третьего этапов.

Пока слой не собран, статус G3 честно описывается одной фразой: условное ядро доказано, компактификация в производстве. Это же — образец инкрементального стиля программы: вместо монолитной теоремы «сингулярность такая-то» поставлены два независимых критерия, один уже соединён с сертификатами, второй ждёт своего слоя. Каждый фрагмент имеет собственный критерий приёмки, и ни один не блокирует остальные маршруты: существование и стабильность не зависят от компактификации.

8.3. Зачем геометрия доказательной программе

Возможно спросить, зачем программе геометрия сингулярности, если её флагманы — существование и масштабирование. Ответ двоякий. Содержательно: сильная и изотропная сингулярность — часть того, что делает критическое решение универсальным аттрактором; без её формализации утверждение о пороге остаётся неполным, и будущие потребители теории — от численной относительности до термодинамики чёрных дыр — получат теорему без её геометрической половины. Технически: геометрический слой тренирует библиотеку на задачах, которых mathlib ещё не знает, и эти тренировки — компактификация, нулевые критерии, ковариационные масштабы — останутся в mathlib как общий актив после того, как частная задача будет закрыта.

В связке с каналами программы геометрия получает и свою логистику: сертификаты коэрцитивности и вейлевских оценок имеют заранее определённую схему и потребителя — условное ядро этой главы. Когда численная ветвь геометрии будет готова их производить, они войдут в теорию по тому же конвейеру, что и спектральные: рациональные границы, сьюты, разрядка. Единообразие конвейера — свойство, которое программа защищает почти идеологически: новых способов входа данных в теорию нет и не будет.

8.4. Ковариационные масштабы: как измерять сингулярность

Утверждение «вейлевские компоненты ограничены» требует указания масштабов: ограничены относительно чего? В критическом решении естественный ответ даёт самоподобность: все величины измеряются в ковариационных масштабах — степенях кривизны, пересчитанных по закону ковариации периода. В этих масштабах ограниченность Вейля — содержательное свойство решения, а не тривиальность выбора единиц; расходимость интеграла силы — тоже: интеграл считается по нулевой геодезической относительно ковариационно нормированной плотности. Формализация фиксирует оба определения явно, и лемма о репараметризационной инвариантности гарантирует, что смена представителя класса самоподобия не меняет диагноза.

Работа с масштабами — место, где формальная математика дисциплинирует физическую интуицию. В литературе о критическом коллапсе утверждения о «сильных» и «слабых» сингулярностях часто опираются на невысказанные соглашения о масштабах; формализация делает соглашения частью определения, после чего спорить о смысле становится невозможно — можно только доказывать или опровергать утверждения в зафиксированных терминах. Это ещё один пример общего правила: всё, что можно сделать определением, не должно оставаться намёком.

8.5. Как G3 обслуживает остальных

Геометрия сингулярности не изолирована от флагманских маршрутов — она их страхует и проверяет. Страхует: если будущие вычисления покажут нарушение вейлевской ограниченности или силы, это будет сигналом о проблеме в самом критическом решении, то есть в данных, на которых стоят маршруты существования и стабильности. Проверяет: сертификаты G3 — коэрцитивность и вейлевские оценки — проходят через тот же конвейер, что и спектральные, и их производство тренировало бы те же muscles численного крыла. Взаимная зависимость маршрутов через общий канал — архитектурное свойство программы, а не случайность.

Есть и обратная услуга: маршруты кормят геометрию. Существование даёт сам объект, спектр — его масштабы, канал сертификатов — способ говорить о них точно.

К моменту, когда численная ветвь геометрии будет готова производить свои сертификаты, у неё будут готовы и поставщики (решатель), и потребитель (условное ядро), и путь в теорию (канал). По опыту двух остальных маршрутов это означает, что сборка G3 после готовности данных займёт не месяцы, а недели — конвейер уже работает.

9. Инфраструктура качества и воспроизводимость

Архитектура доверия живёт не только в теоремах, но и в правилах обращения с деревом. Настоящая глава собирает эти правила воедино: аудит аксиом, дисциплину sorry-тегов, регрессионные замки, непрерывную интеграцию и детерминированную воспроизводимость. Общий принцип один: каждое свойство, на которое опирается доверие, должно быть проверяемо машиной, а не памятью участников; там, где проверка невозможна, должно быть явное ограничение скоупа, а не молчание.

9.1. Аудит аксиом: тридцать три теоремы, три аксиомы

Аудит аксиом — модуль, в котором каждая итоговая теорема программы проходит через команду печати аксиом, а результат сравнивается с ожидаемым множеством из трёх стандартных аксиом. Тридцать три записи — двадцать одна теоретическая и двенадцать мостовых — образуют публичный список того, что программа считает флагманскими результатами; этот список машиночитаем, и CI не даёт ему расходиться с деревом. Любая новая теорема, претендующая на статус итоговой, обязана войти в аудит со своей проверкой аксиом; любая теорема, выпавшая из списка, теряет статус автоматически.

Практический смысл аудита лучше всего виден от противного. Без него доказательная база размывается тихо: новый импорт тянет за собой неявные зависимости, удобная аксиома экономит неделю работы, и через полгода «машинно проверенная» теория опирается на полдюжины заявлений, которых никто не проверял. С ним доверенная база заморожена на уровне ядра и трёх аксиом, разделяемых всей mathlib; любые изменения базы видны в diff'е аудита и обсуждаются как событие, а не происходят как побочный эффект. История программы знает пример обратного хода: три аксиомы, внесённые на раннем каркасе, были позже удалены, и аудит зафиксировал возврат к чистой базе.

9.2. Дисциплина sorry и реестр пробелов

Правило дома гласит: sorry — это запись в реестре, а не удобство. Каждый sorried фрагмент обязан нести в комментарии идентификатор пробела G1–G6, присутствующий в реестре; CI сверяет теги с реестром и роняет сборку при

расхождении. У анонимных пробелов нет места в этой системе: нельзя утаить обязательство, нельзя переименовать его задним числом, нельзя закрыть реестр, пока открыт sorried фрагмент. Сореметрия дерева — публичная метрика, и её история — часть отчётности программы.

Реестр пробелов, в свою очередь, — не список желаний, а таблица обязательств: у каждой строки есть категория, серьёзность, зависимость, стратегия закрытия, этап и статус. Статусы обновляются тем же языком свидетельств, что и вехи: «partial» с перечислением доказанных фрагментов, «resolved» со ссылкой на приёмочное свидетельство, «live» с указанием работающего конвейера. Такая бухгалтерия делает состояние программы воспроизводимым фактом: любой читатель дерева может пересчитать статусы сам, и именно это мы делали во второй главе.

9.3. Непрерывная интеграция

Шаблон CI репозитория запускает три проверки. Первая — сборка Lean с кэшем mathlib: каждая теорема тип-чекается заново, аудит аксиом выполняется на свежей сборке, наличие sorried фрагментов сверяется с реестром. Вторая — pytest-набор численного крыла: шестьдесят пять тестов трёх семейств, включая живой round-trip моста там, где транспорт доступен. Третья — регенерационная проверка данных: CSV, рисунки и анимации пересоздаются из фиксированного зерна и сравниваются побайтно; расхождение означает, что кто-то отредактировал генерируемое вручную, и такое изменение не проходит. Три проверки вместе образуют замкнутый контур: теория, мост и данные не могут разойтись незаметно.

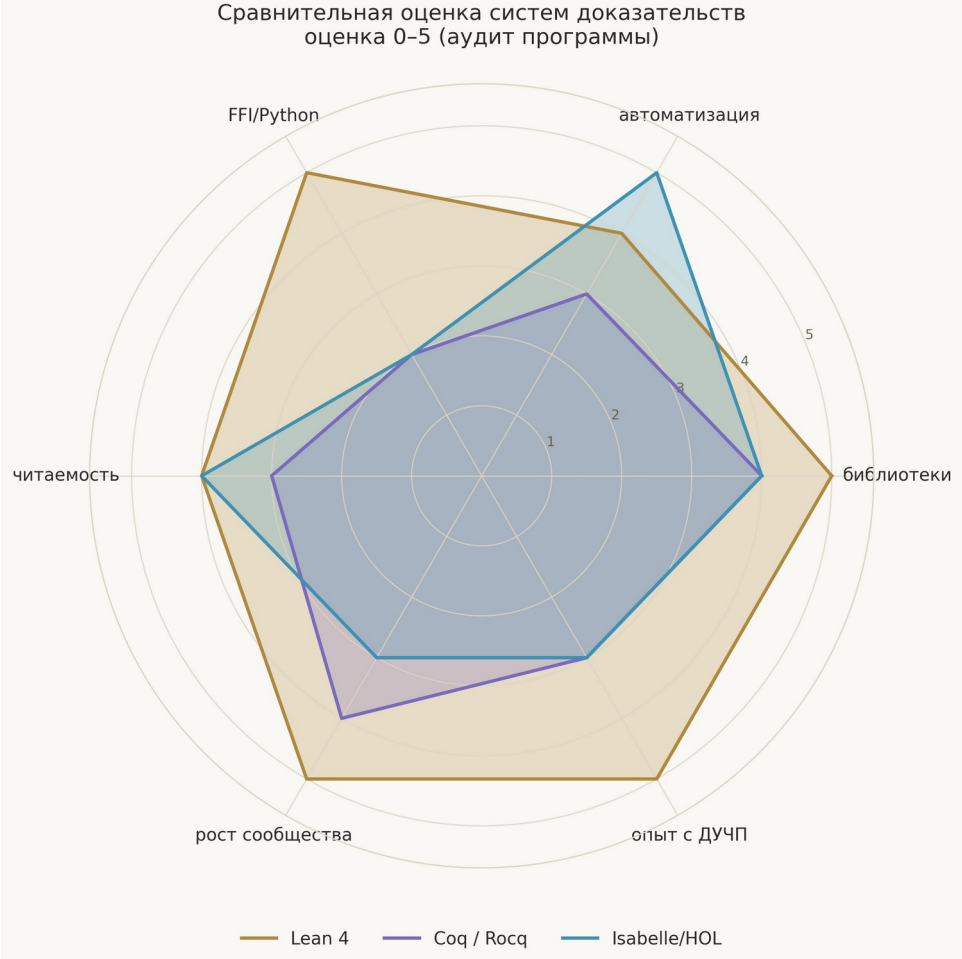


Рис. 9. Радар инструментов: оценка кандидатов-доказателей по критериям программы — логика, библиотека, автоматизация, интеграция с Python, зрелость экосистемы; выбор Lean 4 зафиксирован в решающей матрице тома II.

9.4. Воспроизводимость от единственного зерна

Всё численное содержимое дерева — данные, рисунки, анимации, сертификаты — регенерируется из одного фиксированного зерна, 1993 года открытия критических явлений. Зерно — не суеверие, а дизайн: никакой случайности в рендере нет, генераторы детерминированы, и одинаковый вход даёт одинаковый выход на любой корректной платформе. Сертификаты, кроме того, байт-стабильны на любой корректной реализации LAPACK — это более сильное свойство, чем детерминизм одного процесса, и оно специально заперто тестами, потому что на нём держится цитируемость артефактов.

Воспроизводимость распространяется и на тексты: рисунки монографий существуют в двух темах и двух языках и генерируются одними и теми же скриптами с разными флагами; данные под каждым рисунком коммитятся рядом с ним; каждый рисунок снабжён подписью с указанием зерна. Читатель третьего тома может воспроизвести любую его иллюстрацию, не доверяя ни автору, ни издателю, — и это

не побочная функция, а продолжение той же дисциплины проверки, что и в ядре доказателя.

Таблица 5. Правила дома — сводка

Правило	Механизм исполнения
sorry — запись в реестре	тег G1–G6 в комментарии, CI-сверка с реестром пробелов
никаких новых проектных аксиом	CI-grep деклараций axiom в теоретических модулях, аудит аксиом 33 итоговых теорем
изменение протокола только с парным тестом	конвенция репозитория + pytest-замки схем каждого метода
генерируемое не редактируется руками	байт-идентичная регенерация из seed 1993 в CI
каждая математическая идея сначала — проза	blueprint-документы и комментарии о роли фрагмента в дереве доказательства

9.5. Сводные метрики доверия

Соберём метрики, по которым программа измеряет себя, в одну таблицу. Все они пересчитываемы из дерева за минуты, и каждая привязана к механизму исполнения — потому что метрика без механизма превращается в лозунг. Эту таблицу можно использовать как чеклист при ревизии репозитория: если какая-то строка не воспроизводится, программа знает об этом раньше любого внешнего проверяющего.

Таблица 11. Метрики доверия программы

Метрика	Значение	Механизм
открытые обязательства (sorry)	0	grep по дереву + реестр пробелов
проектные аксиомы	0	CI-grep деклараций axiom
итоговые теоремы с аудитом аксиом	33	модуль аудита, три стандартные аксиомы
pytest-тесты	65	три семейства: сервис, сертификаты, клиент
видов сертификатов	4	закрытый перечень, bad_input на иное
детерминированные CSV	7	seed 1993, регенерация в CI
строк данных	12 890	подсчёт по файлам, регенерационный тест
иллюстрации	8 PNG @ 300 dpi + 4 GIF	генераторы с флагами темы и языка

Ни одна строка таблицы не является «целью, к которой идём» — все они являются свойствами текущего дерева, проверяемыми сегодня. Это принципиально: метрики доверия в программе описывают наличное состояние, а не амбиции, потому что смешение этих двух регистров — стандартный способ, которым технические отчёты вводят в заблуждение. Амбиции тоже присутствуют в программе, но у них своё место — дорожная карта с датами и критериями приёмки, — и они никогда не смешиваются с инвентаризацией.

9.6. Воспроизводимость этого тома

Том проходит через ту же дисциплину, что описывает. Все его иллюстрации сгенерированы скриптами репозитория из коммитимых данных в двух темах и двух языках; диаграмма канала сертификатов воспроизводится отдельным скриптом с фиксированной геометрией; числа взяты из README, реестров и тестов и перепроверены подсчётом. Текст доступен в двух языках и двух форматах, собранных из одного источника, — так что расхождение между версиями исключено по построению, а не по честному слову.

Есть и второй уровень воспроизводимости — сюжетный. Каждое утверждение тома снабжено указанием, где оно живёт в дереве: модуль, протокол, реестр, тест. Читатель может пройти по этим указаниям и за один вечер пересобрать всю цепочку от сертификата до разрядки на собственном компьютере; для этого нужны Lean-тулчейн с mathlib и Python с numpy — оба стеки открыты, обе сборки автоматизированы. Мы считаем это лучшей формой доверия к научному тексту: не «поверьте нам», а «вот команда, которой это проверяется».

10. Заключение: архитектура доверия

Третий том серии был посвящён тому, как программа formal_choptuik превращает вычисления в доказуемые утверждения, не подмешивая веры ни в одной точке пути. Соберём итог в одной картинке. Интервальная арифметика с доказанной корректностью даёт теории язык строгих оценок; мост с семью методами даёт исследователю руки, а каналу сертификатов — дорогу; конвейер сертификатов несёт в теорию точные рациональные данные, проверенные ядром; теоремы робастности переводят проверенные данные в физические утверждения; разрядки делают утверждения безусловными; аудит аксиом гарантирует, что всё это стоит ровно на трёх стандартных аксиомах и ядре. Ниже этой конструкции — вычисления, над ней — физика критических явлений; между ними нет ни одного непроверенного шага.

10.1. Что меняет канал сертификатов методологически

Первое изменение — эпистемологическое: граница между свидетельством и доказательством перестала быть размытой. В классической вычислительной физике утверждение живёт на шкале доверия: подтверждено тестами, воспроизведено группами, согласовано с наблюдениями. В архитектуре сертификатов у утверждения есть точный статус: либо оно разряжено из проверенного сертификата и является теоремой, либо оно численное свидетельство с известной оценкой, либо его нет. Промежуточных состояний, в которых число «почти теорема», архитектура не оставляет — и этим защищает и доказательства от недоказанного, и численность от притязаний, которые она не может выполнить.

Второе изменение — организационное: роли proof-инфраструктуры и численной инфраструктуры разделены, но симметричны. Численное крыло отвечает за тесноту оценок и физическую адекватность модели; доказательное — за то, чтобы каждое принятое утверждение следовало из проверенных посылок; мост — за честность переноса. Ответственность каждой стороны выражена в тестах и теоремах, а не в договорённостях. Именно эта симметрия делает программу живой системой: она не требует героической бдительности и ловит собственные расхождения автоматически — как показал эпизод с латентными дефектами протокола и как ежедневно показывают регрессионные замки.

Третье изменение — временно́е: инфраструктура, построенная заранее, конвертируется в скорость будущих доказательств. Все три маршрута программы — существование, стабильность, геометрия — используют один и тот же конвейер, и когда будут готовы их сертификаты, ни одна строчка инфраструктуры не потребует изменений. Спектральный сертификат истинного оператора войдёт в теорию так же, как вошёл модельный: рациональные границы, сьюты, робастная теорема, разрядка. В этом смысле том, который вы читаете, — не отчёт о завершении, а инвентаризация трамплина.

10.2. Дорога вперёд

Ближайшая дорога программы — вторая очередь: формализация системы поля за каркасом, упаковка численного зерна высокого разрешения как Lean-данных, банахово пространство и компактный обратный, сборка существования. Критерий успеха зафиксирован: печать аксиом у critical_solution_exists показывает три стандартные аксиомы, период эха построенного решения совпадает с сервисным до сертифицированной точности. Третья очередь — линеаризация и спектральная теория на истинном операторе, разрядка unique_unstable_mode, вывод γ с допуском к опубликованному значению, сборка универсальности. Геометрический слой — компактификация — идёт параллельно, не блокируя флагманские маршруты.

Риски маршрутов известны и расписаны: компактный обратный в Lean — самая тяжёлая аналитическая работа, с запасным путём через формализацию опубликованных неравенств; спектральная теория неограниченных операторов — самая тяжёлая библиотечная зависимость, со скоупом на конкретный оператор; упаковка численного зерна — задача, решаемая каналом сертификатов, построенным именно для этого. Ни один из рисков не является неизвестным: у каждого есть стратегия, критерий приёмки и место в реестре. Программа может не успеть — но она не может заблудиться.

10.3. Уроки для машинно проверяемой физики

Опыт, собранный в этом томе, обобщается за пределы чоптуиковской тематики, и мы сформулируем его как три урока. Первый: границу надо проектировать раньше содержимого. Программа потратила заметную часть первого этапа на вещи, которые в статье физики заняли бы сноску, — на округление наружу, схемы ответов, коды ошибок, таймауты, — и именно эта работа сделала остальное проверяемым. Второй: гипотеза, вынесенная в интерфейс, лучше аксиомы, спрятанной в доказательстве. Робастная форма теорем — спектральных, ньютоновских, геометрических — позволяет развивать аналитический контур, не дожидаясь вычислительного содержимого, и при этом никогда не путать контур с содержимым. Третий: воспроизводимость — это форма доверия, доступная машине; всё, что можно сделать байт-стабильным и детерминированным, должно быть сделано так, потому что всё остальное придётся принимать на веру.

Чоптуиковская программа началась с трёх чисел, которые обязаны быть доказанными: показатель γ, период Δ, коразмерность один. К моменту написания этого тома два из трёх сюжетов имеют полную доказательную инфраструктуру и модельные разрядки, третий — условное ядро; вся конструкция стоит на ядре доказателя и трёх аксиомах, которые оно приносит с собой. Оставшаяся работа — содержательная физика: сертификаты для истинной системы. Она большая; но теперь она — единственное, что осталось, и у каждого её куска есть готовый приёмник в теории. Именно этого программа и обещала: не ускорить путь к теореме ценой строгости, а убрать из пути всё, кроме самой теоремы.

10.4. Сравнение с альтернативными архитектурами

Архитектуру сертификатов полезно сравнить с альтернативами, чтобы видеть цену решения. Альтернатива первая — доверенный вычислитель: ядро доказателя само считает числа, и доверие к результату сводится к доверию к ядру. Схема элегантна, но для физики непрактична: плотные линейно-алгебраические вычисления в тактиках доказателя на порядки медленнее LAPACK, а их формализация —

отдельная программа на годы. Альтернатива вторая — интерактивная верификация: доказатель и вычислитель переписываются по сети в реальном времени, и каждый шаг вычисления проверяется. Схема честная, но доверенная база вырастает на весь транспорт и весь вычислитель одновременно; кроме того, она ставит доказательство в зависимость от живого процесса — реплицируемость падает.

Выбранная третья схема — офлайн-сертификаты — занимает промежуточную позицию с точной асимптотикой доверия: доверенная база остаётся ядро плюс три аксиомы, вычислитель полностью вне её, транспорт сведён к коммитимым файлам, реплицируемость полная. Плата — дисциплина производства: каждый новый тип данных требует схемы, сьютов и разрядки. Программа считает эту цену правильной покупкой: она платится один раз на слой данных, а альтернативы платят непрерывно — скоростью, доверием или воспроизводимостью. История вычислительной математики последних десятилетий — целочисленные доказательства, SAT-верификация, формализованные библиотеки — движется в ту же сторону.

10.5. Приглашение к проверке

Заключим том тем, с чего он начался, — отношением к читателю. Всё, что здесь описано, проверяемо без всякого доверия к автору: дерево собирается, тесты запускаются, теоремы печатают свои аксиомы, сертификаты регенерируются байт-в-байт, иллюстрации строятся из данных на глазах. Команды для этого короткие и собраны в README репозитория; полный цикл проверки — от lake build до pytest и certify check — умещается в один сеанс за терминалом. Тот, кто найдёт расхождение между этим текстом и деревом, найдёт дефект в дереве или в тексте — оба случая для программы ценны, и оба исправляются по одним и тем же правилам свидетельств.

Именно в этом мы видим главный итог архитектуры доверия: она переводит научную коммуникацию из регистра заверений в регистр проверок. Между автором и читателем больше нет асимметрии, которую нельзя устранить машиной; осталась только работа, которую машина берёт на себя. Третий том серии — описание этой машины: её конструкции, её гарантий и её цены. Дальше — физика, и мы рассчитываем вернуться к ней в следующих томах уже в форме, известной этому тому: сертификат, проверка, разрядка, теорема.

Приложение A. Глоссарий программы

Глоссарий собран по образцу справочной полки репозитория и адаптирован для читателя-новичка в доказательных ассистентах. Семнадцать терминов ниже покрывают весь словарь тома; каждое определение даётся в том значении, в котором термин используется в формализации, а не в самом широком математическом смысле.

Термины, отмеченные пробелами реестра, связаны с соответствующими строками G1–G6.

Таблица 12. Семнадцать терминов программы

Термин	Значение
критический коллапс	гравитационный коллапс безмассового скалярного поля точно на пороге образования чёрной дыры p = p* (Чоптуик, 1993)
ДСС — дискретная самоподобность	сдвиг логарифмического времени на Δ равносилен пересчёту радиуса площади на e^{−Δ} с амплитудой e^{кΔ}
период эха Δ	лог-временной период ДСС-решения; численно Δ ≈ ln 2 ≈ 0.693
лог-время τ	τ = −ln(T − t) — временна́я координата у сингулярности, в которой эхо периодично
γ — критический показатель	показатель масштабирования массы, M ∝ |p − p*|^γ; γ = 1/λ_0 ≈ 0.374
λ_0	единственное неустойчивое собственное значение оператора, линеаризованного на критическом решении
межэховое расстояние	мнимые части семейства возмущений отстоят на 2π/Δ ≈ 9.06 — спектральное эхо
аттрактор коразмерности один	критическое решение притягивает все близкие эволюции, кроме одного направления — растущей моды
изотропная сингулярность	конец, где инварианты кривизны расходятся, а компоненты Вейля остаются ограниченными; допускает конформную компактификацию (G3)
аспект массы	величина в двойных нулевых координатах, чей ненулевой предел на нулевой бесконечности означает чёрную дыру
интервал	замкнутый отрезок [lo, hi]; внешне-округлённые операции делают оценки звукими (G5)
сертификатный принцип	если свойство выполнено на интервальной оценке, оно выполнено для любого значения внутри неё — мост из численности в доказательство
блокирующий пробел	пробел (G1 — существование; G2 — стабильность), без которого ни одно внешнее утверждение программы не проверяемо машиной
sorry (Lean)	явная заглушка доказательства; каждый sorry в lean/Choptuik — запись реестра пробелов
mathlib	математическая библиотека Lean — топология, функциональный анализ, многообразия; субстрат формализации
ZeroMQ-мост	JSON-канал REQ/REP, позволяющий Lean управлять Python-сервисом (см. протокол)
sorry как запись реестра	правило дома: заглушка без идентификатора пробела недопустима; CI сверяет теги с реестром

Приложение Б. Карта модулей Lean

Теоретическое дерево организовано как пространство имён Choptuik.*: одна математическая область — один модуль; каждый модуль компилируется независимо против mathlib и своих импортов. Карта ниже связывает модуль, его содержание и строки реестра пробелов; она повторяет машиночитаемую карту репозитория и служит путеводителем по сборке. Корневой файл реэкспортирует пространство, отдельный модуль аудита печатает аксиомы всех итоговых теорем.

Таблица 13. Модули теории и их содержание

Модуль	Содержание	Пробел	Этап
Geometry.Basic	арена двойных нулевых координат, положительность радиуса площади, аспект массы	—	I
Geometry.Curvature	профили Кречмана и Вейля, сильная и изотропная сингулярность	G3	II–III
Analysis.HyperbolicPDE	система Эйнштейна–скаляра, ограничения, корректность	G1	II
Analysis.Interval	тип Interval, корректность операций, сертификатный принцип	G5	I
Existence.DSS	ковариация ДСС, критическое решение, существование и единственность	G1	II
Existence.ImplicitFunction	банахов маршрут IFT, устойчивость к возмущениям	G1, G5	II
Stability.Linearization	моды возмущений, интерфейс линеаризованного оператора	G2	III
Stability.Spectrum	единственная неустойчивая мода, $\gamma = 1/\lambda_0$, закон масштабирования	G2	III
Scaling	универсальность, формула эха	G2	III
Bridge.Certificates	тип RatIv, интерпретации проверок, леммы робастности	G4	I
Bridge.Generated	точные рациональные данные сертификатов, check-сьюты	G4	I
Bridge.Discharge	разрядка в спектральные, ньютоновские и интервальные итоговые теоремы	G4	I

Граф импортов повторяет логику зависимости результатов: арена (Geometry.Basic) питает и кривизну, и систему поля; интервальный слой обслуживает маршрут неявной функции; ДСС соединяет систему поля с линеаризацией; спектр и масштаб замыкают физическую часть; мост подключается ко всем маршрутам как

канал данных. Каждый пробел реестра привязан к конкретным модулям, и статус строки реестра всегда можно проверить сборкой соответствующего поддерева.

Сборка воспроизводится двумя командами: получение кэшированных артефактов mathlib и собственно сборка, тип-чекающая каждое доказательство и выпускающая объектные файлы. Инструментальная цепочка и ревизия mathlib двигаются вместе — правило закреплено в конфигурации пакета и продублировано в шаблоне непрерывной интеграции. Аудит аксиом запускается отдельной командой поверх собранного дерева и печатает набор аксиом каждой из тридцати трёх итоговых теорем — тот самый список, который разобран в главе 9.

Таблица 14. Карта потребления сертификатов: производитель → проверка → итоговая теорема

Вид	Путь в Lean	Итоговая разрядка
spectrum	производитель → Generated: spectrumChecks → лемма о форме спектра	bridge_unique_unstable_mode
newton	производитель → Generated: newtonChecks → полоса уравнения поправки	correction_root_exists, correction_root_in_band
scaling	производитель → Generated: scalingChecks → согласие с $1/\lambda_0$	scaling_agreement
interval	производитель → Generated: intervalChecks → звукость образа	correction_band_image_sound

Карта потребления — самая короткая сводка канала: четыре строки, и в каждой три звена. Для читателя, впервые открывающего дерево, она же — маршрут экскурсии: открыть сертификат в папке артефактов, найти соответствующий сьют в Generated-модуле, закончить разрядкой и распечатать её аксиомы. Вся экскурсия занимает минуты и проходит без единого обращения к доверию — только к сборке.

Приложение B. Как проверить этот том за один сеанс

Приложение собирает команды полной проверки в один чеклист — от сборки теории до регенерации иллюстраций. Каждая строка — самостоятельный шаг; отказ любого из них сигнализирует о рассинхронизации дерева и текста. Полный цикл на обычной машине занимает от нескольких минут до получаса, большая часть времени уходит на первую загрузку кэша библиотеки.

Таблица 15. Чеклист проверки

Шаг	Команда	Ожидаемый результат
сборка теории	cd lean && lake exe cache get! && lake build	зелёная сборка, объектные файлы всех модулей
аудит аксиом	cd lean && lake env lean AxiomAudit.lean	33 теоремы, три стандартные аксиомы
тесты сервиса	cd python && python -m pytest tests/ -v	65 зелёных тестов
сертификаты: проверка	cd python && python -m choptuik_service.certify check	коммитимые артефакты совпадают с регенерацией
сертификаты: пересоздание	cd python && python -m choptuik_service.certify emit	байт-идентичные JSON и Generated-модуль
живой мост	python choptuik_service/server.py, затем клиент	round-trip по семи методам
данные	python scripts/generate_data.py	CSV байт-в-байт совпадают с коммитом
рисунки	python scripts/generate_figures.py --theme light --lang ru	PNG совпадают с используемыми в томе
анимации	python scripts/generate_animations.py	GIF регенерируются из seed 1993

Проверка тома завершается сверкой двух списков: списка итоговых разрядок главы 5 и распечаток аксиом шага два; они должны совпасть поэлементно. Для читателя, желающего проверить не только факты, но и иллюстрации, флаги темы и языка генераторов воспроизводят каждую картинку тома — светлая тема и русский или английский язык для рисунков, тёмная для обложечных баннеров репозитория. Успешное прохождение чеклиста означает, что каждое утверждение этого тома воспроизводится на вашей машине без участия автора — что и было целью архитектуры доверия.

Приложение Г. Реестр итоговых теорем и вех

Реестр ниже перечисляет все тридцать три итоговые теоремы аудита аксиом — двадцать одну теоретическую и двенадцать мостовых — с однострочным содержанием каждой. Это тот самый список, который печатается модулем аудита при сборке; каждая строка таблицы соответствует строке аудита, и расхождение между ними невозможно по построению. Для полноты рядом приведён реестр из девяти вех с их статусами — календарный каркас программы.

Таблица 16. Тридцать три итоговые теоремы (аудит аксиом)

Область	Теорема	Содержание
Geometry	divergence_invariant_under_rescaling	инвариантность критерия силы при репараметризации (G3)
Geometry	critical_singularity_is_strong_and_isotropic	условное ядро: сертификаты влекут сильную и изотропную сингулярность (G3)
Analysis	add_sound	корректность сложения интервалов (G5)
Analysis	mul_sound	корректность умножения интервалов (G5)
Analysis	certificate_principle	проверенная оценка есть теорема (G5)
Analysis	local_wellposedness	локальная корректность системы поля, слабая форма (G1)
Existence	critical_solution_exists	каркасное существование критического решения (G1)
Existence	dss_fundamental_domain_determination	определение фундаментальной области ДСС (G1)
Existence	critical_solution_unique	единственность с точностью до масштаба и сдвига (G1)
Existence	dss_reparametrization_invariance	репараметризационная инвариантность ДСС (G1)
Existence	newton_map_contractive	сжимаемость ньютоновского отображения (G1)
Existence	existence_via_implicit_function	существование через количественную теорему о неявной функции (G1)
Existence	newton_zero_unique	единственность нуля в шаре (G1)
Existence	existence_persists_under_perturbation	устойчивость существования к возмущениям носителя (G1)
Existence	derivIsomorphism_iff	критерий изоморфизма производных для маршрута IFT (G1, G5)
Stability	unique_unstable_mode	единственная неустойчивая мода, при сертификатной гипотезе (G2)
Stability	echo_family_structure	структура семейства эхо-сдвинутых решений (G2)
Stability	mass_scaling_law	закон масштабирования в лог-лог форме (G2)
Scaling	gamma_close_to_published	выведенный показатель в допуске к калиброванному (G2)
Scaling	universality	универсальность в условной форме (G2)
Scaling	echoing_formula	формула эха (G2)
Bridge	im_odd_multiple	мнимые части затухающих мод кратны $2\pi/\Delta$ (G4, G2)

Область	Теорема	Содержание
Bridge	spectrum_shape_of_certificate	форма спектра, выводимая из сертификата (G4, G2)
Bridge	newton_kantorovich_local	локальная схема Ньютона–Канторовича при сертификатных константах (G4, G1)
Bridge	spectrumChecks_generated	сьют проверки спектрального сертификата (G4)
Bridge	newtonChecks_generated	сьют проверки ньютоновского сертификата (G4)
Bridge	scalingChecks_generated	сьют проверки масштабного сертификата (G4)
Bridge	intervalChecks_generated	сьют проверки интервального сертификата (G4)
Bridge	bridge_unique_unstable_mode	разрядка: единственная неустойчивая мода модели (G4, G2)
Bridge	correction_root_exists	разрядка: существование корня уравнения поправки (G4, G1)
Bridge	correction_root_in_band	разрядка: корень лежит в проверенной полосе (G4, G1)
Bridge	correction_band_image_sound	разрядка: звукость образа полосы (G4, G5)
Bridge	scaling_agreement	разрядка: согласие подгонки с $1/\lambda_0$ в допуске (G4, G2)

Таблица 17. Реестр вех программы

Веха	Месяц	Содержание	Статус
M1.1	1	рабочее окружение операционально	выполнена
M1.2	5	интервальные сертификаты верифицированы	выполнена
M1.3	6	мост жив: типизированные round-trip'ы и канал сертификатов	частично: канал сертификатов жив, FFI-клиент остаётся иллюстративным
M2.1	10	система поля формализована	открыта
M2.2	12	численное зерно упаковано как данные Lean	открыта
M2.3	18	существование доказано	открыта; каркас и NK-ядро готовы, сертификат PT ждёт
M3.1	23	спектральное ядро доказано	открыта; доказано при сертификатной гипотезе

Веха	Месяц	Содержание	Статус
M3.2	25	закон масштабирования выведен	открыта; log-лог форма и согласие показателя доказаны
M3.3	27	универсальность собрана	открыта

Совмещение двух таблиц — теорем и вех — даёт полную картину состояния: какие высказывания уже стоят на трёх аксиомах и какие календарные события отделяют текущее состояние от финала. Веха, напомним, меняет статус только при закоммиченном приёмочном свидетельстве; поэтому статусы таблицы всегда пересчитываемы из дерева — что и рекомендуется сделать по чеклисту приложения B.

Приложение Д. Соответствие глав тома и дерева

Последняя сводка связывает главы тома с файлами дерева, которые они описывают, — для читателя, идущего параллельно по тексту и по репозиторию. Каждая строка называет точки входа соответствующей главы; все они перечислены в карте папок (раздел 2.8), а здесь собраны по сюжетам.

Таблица 18. Главы тома и соответствующие файлы дерева

Глава	Файлы дерева
2. Состояние программы	README.md, roadmap/milestones.csv, blueprint/gap_registry.csv, data/
3. Интервальная арифметика	lean/Choptuik/Analysis/Interval.lean, python/choptuik_service/interval_checks.py
4. Мост	bridge/protocol.md, python/choptuik_service/client.py, python/choptuik_service/server.py, bridge/examples/LeanClient.lean
5. Канал сертификатов	python/choptuik_service/certify.py, bridge/certificates/, lean/Choptuik/Bridge/Generated.lean, lean/Choptuik/Bridge/Certificates.lean, lean/Choptuik/Bridge/Discharge.lean
6. Существование	lean/Choptuik/Existence/DSS.lean, lean/Choptuik/Existence/ImplicitFunction.lean, lean/Choptuik/Analysis/HyperbolicPDE.lean
7. Стабильность	lean/Choptuik/Stability/Linearization.lean, lean/Choptuik/Stability/Spectrum.lean, lean/Choptuik/Scaling.lean
8. Сингулярность	lean/Choptuik/Geometry/Basic.lean, lean/Choptuik/Geometry/Curvature.lean

Глава	Файлы дерева
9. Качество	lean/AxiomAudit.lean, ci/lean-ci.yml, blueprint/technical_specification.md, blueprint/axiomatic_base.md
Приложение А	docs/glossary.md
Приложение Б	blueprint/module_map.md, lean/Choptuik.lean, lean/lakefile.toml

Чтение параллельно с деревом — лучший способ освоить программу: каждая глава становится на порядок конкретнее, когда её утверждения видны в исходниках и в тестах. Мы сознательно не приводили в томе листинги кода: код живёт в репозитории, где он компилируется и проверяется, а том отвечает за карту, по которой к этому коду можно подойти без блужданий.

Библиография

[1] Choptuik, M. W. Critical behaviour in gravitational collapse of a scalar field. Physical Review Letters, 1993.

[2] Gundlach, C., Martin-Garcia, J. M. Critical phenomena in gravitational collapse. Living Reviews in Relativity, 2007.

[3] Abrahams, A. M., Evans, C. R. Universality and scaling in gravitational collapse of a massless scalar field. Physical Review Letters, 1993.

[4] Evans, C. R., Coleman, J. S. Critical phenomena and self-similarity in the gravitational collapse of radiation fluid. Physical Review Letters, 1994.

[5] Koike, T., Hara, T., Adachi, S. Critical behavior in gravitational collapse. Physical Review Letters, 1995.

[6] Eardley, D. M., Giddings, S. B. Analytical black-hole formation. Physical Review D, 2002.

[7] Reiterer, M., Trubowitz, E. The Choptuik critical solution. Communications in Mathematical Physics / arXiv preprints, 2019.

[8] de Moura, L., Kong, S., Avigad, J., van Doorn, F., von Raumer, J. The Lean theorem prover (system description). CADE, 2015.

[9] The mathlib Community. The Lean mathematical library. CPP, 2020.

[10] Moore, R. E. Interval Analysis. Prentice-Hall, 1966.

[11] Kantorovich, L. V., Akilov, G. P. Functional Analysis. Pergamon Press, 1982.

[12] Isaev, I. K. choptuik_ac_bc: machine-verified research programme (parent repository). GitHub, 2026.

[13] Isaev, I. K. formal_choptuik: from code to machine-checked proof — Volume I of the series. GitHub, 2026.

[14] Isaev, I. K. Gap analysis and proof-assistant selection — Volume II of the series. GitHub, 2026.